Stanford CS224n《NLP with Deep Learning》学习笔记

课程：Stanford CS224N / Ling284 *Natural Language Processing with Deep Learning*（当季主页标注 "Stanford / Winter 2026"） **课程主页**：web.stanford.edu/class/cs224n；日程锚点：`index.html#schedule` **讲义**：本笔记逐讲阅读的是当季公开 PDF，路径形如 `https://web.stanford.edu/class/cs224n/slides_w26/cs224n-2026-lectureNN-<topic>.pdf`（共 17 份可下载讲义，覆盖 19 个讲次中的 16 个；3 个客座讲次未公开当季幻灯片，见 1.6 节） **教材**：Jurafsky & Martin, *Speech and Language Processing*, 3rd ed. draft，免费在线：web.stanford.edu/~jurafsky/slp3。主页抓取到的是 2026-08-19 版合订本，章节编号与 2024 版不同（见 25.4 节末的编号提示） **作业**：A1–A4 共 4 个（48%）+ 期末项目（49%）+ 参与分（3%）；起始代码与 handout 均在课程主页的 `assignments_w26/`、`project_w25/` 下公开 **视频**：当季录像走 Canvas/Panopto，仅注册学生可见；**公开完整视频**为 2024 版 YouTube 播放列表 `PLoROMvodv4rOaMFbaqxPDoLWjDaRAdP9D` **授课教师（当季主页列出）**：Diyi Yang、Yejin Choi；Head TA：Julie Kallini；Course Manager：John Cho **材料抓取日期**：2026-11-02。所有具体数字（评分比例、讲次日期、模型超参、baseline 准确率、benchmark 分数）均来自该日抓取到的公开 PDF/HTML 原文；凡属我外推的，正文标"推断" **免责**：评分比例、讲次顺序、作业题号**每学期都会变**；关于作业与项目的任何决定请以当学期主页为准 **定位**：本笔记是"用 Transformer 解决 NLP 任务"的**建模视角**——讲清每个模型的动机、目标函数、训练细节、实验现象与常见误区；与同目录 `CS336_学习笔记.md` 的**系统视角**（tokenizer 实现、FLOPs、并行、kernel、scaling law 拟合）互补，不重复造 tokenizer/训练框架。数学基础（偏差–方差、正则化、凸优化）见 `CS229_Machine_Learning_Notes.md`

1.1 一句话定位
1.2 先修（主页明示）
1.3 评分与政策（当季原文数字）
1.4 讲次地图（Winter 2026，19 个讲次）
1.5 与 CS336、CS229 的分工
1.6 材料可访问性与本笔记的证据边界
2.1 动机：历史不是装饰，是"假设决定上限"的证据
2.2 四个时代与代表性"模型"
2.3 关键事件的"公式化"读法
2.4 训练细节：数据与算力的共同演化
2.5 实验现象：深度学习第一次赢在哪
2.6 常见误区
3.1 动机：one-hot 的两个致命问题
3.2 模型公式：skip-gram 与 softmax
3.3 训练细节：负采样与高频词下采样
3.4 实验现象：类比、相似度与"线性结构"
3.5 常见误区
4.1 动机：从"线性分类器"到"可学习的表示"
4.2 模型公式：从 score 到 cross-entropy
4.3 训练细节：雅可比、shape convention 与计算图
4.4 实验现象：为什么手推梯度仍然值得学
4.5 常见误区
5.1 动机：一切生成任务的底座
5.2 模型公式：n-gram 与马尔可夫假设
5.3 模型公式：从固定窗口到 RNN
5.4 训练细节：梯度消失/爆炸与 LSTM
5.5 实验现象：从 LM 到 seq2seq 与瓶颈
5.6 常见误区
6.1 动机：瓶颈与"直连"
6.2 模型公式：从 cross-attention 到 self-attention
6.3 训练细节：位置、非线性、mask、残差与 LayerNorm
6.4 实验现象：更快、更好，但二次代价
6.5 常见误区
7.1 动机：从"做作业"到"做研究"
7.2 模型公式：项目的两种形态
7.3 训练细节：实验方法论（这一节比任何模型都重要）
7.4 实验现象：什么样的项目拿高分
7.5 常见误区
8.1 动机：从"预训练词向量"到"预训练整个模型"
8.2 模型公式：三种架构 × 三种目标
8.3 训练细节：规模、算力与"预训练一次，微调多次"
8.4 实验现象：预训练到底教会了什么
8.5 常见误区
9.1 动机：语言建模 ≠ 助手
9.2 模型公式：从奖励到 DPO
9.3 训练细节：三阶段流水线与数据来源
9.4 实验现象：RLHF 到底改变了什么
9.5 常见误区

10.1 动机：全参微调不现实
10.2 模型公式：LoRA 的低秩分解
10.3 训练细节：容量、秩与"什么时候有效"
10.4 实验现象：提示学习的天花板与不稳定性
10.5 常见误区
11.1 动机：模型不该记住一切
11.2 模型公式：retriever–reader 与 RAG
11.3 训练细节：长上下文的"中间迷失"
11.4 模型公式与现象：agent 的四件套
11.5 常见误区
12.1 动机：评测是"定义问题"的一部分
12.2 基准的演化与设计准则
12.3 指标：从 n-gram 到模型评判
12.4 人类评测与 LLM-as-judge
12.5 数据污染与"生成器-验证器差距"
12.6 常见误区
13.1 动机：推理能力是"训出来的"还是"解码出来的"
13.2 模型公式：从贪心到采样，再到 GRPO
13.3 训练细节：R1-Zero → R1 → R1-distill
13.4 实验现象：PPO 的复杂度与"推理的涌现"
13.5 常见误区
14.1 动机：推理的成本结构
14.2 模型公式：投机解码与同策略蒸馏
14.3 训练细节：长上下文扩展
14.4 实验现象：test-time compute 的收益
14.5 常见误区
15.1 动机：tokenizer 是"模型的感官"
15.2 三种分词方案的取舍
15.3 训练细节：分词器与模型数据必须一致
15.4 实验现象：拼写、多语言与不公平
15.5 常见误区
16.1 动机：能力与可靠性不同步
16.2 幻觉的两种解释
16.3 创造力的悖论
16.4 对齐的挑战
16.5 常见误区
17.1 动机：算力受限下还能不能做前沿
17.2 方法要点：ProRL 与"熵可控"的裁剪
17.3 训练细节与实验现象
17.4 三场无公开讲义的客座（据讲题与公开读物概述）
17.5 课程层面留下的开放问题
17.6 常见误区
18.1 skip-gram 的梯度：为什么是 $U(\hat{y}-y)$
18.2 负采样的梯度与采样分布
18.3 GloVe：从"共现概率比"到加权最小二乘
18.4 共现矩阵 + SVD：A1 Part 1 的完整流程
18.5 评估协议：内在、外在、以及与人类判断的相关
18.6 词向量里的偏见：机制与缓解
18.7 常见误区（专题补充）
19.1 RNN 的 BPTT 与雅可比连乘
19.2 梯度裁剪：公式与实现
19.3 LSTM 的梯度通路
19.4 四种"序列建模"路线的对照
19.5 调试序列模型的检查清单
19.6 常见误区（专题补充）
20.1 一张维度表（以 GPT-2 small / BERT-base / Llama-3-8B 为例）
20.2 复杂度与显存
20.3 Pre-LN vs Post-LN
20.4 位置编码的演化
20.5 一个可运行的 attention 骨架（含形状注释）
20.6 常见误区（专题补充）
21.1 三种架构的对照（含目标函数与典型用法）
21.2 微调配方（经验值，需按任务调）
21.3 灾难性遗忘与"微调把模型弄坏"
21.4 PPO / DPO / GRPO 的选型表
21.5 常见误区（专题补充）
22.1 困惑度：定义、口径与陷阱
22.2 BLEU：公式 + 手算例子
22.3 ROUGE：召回视角
22.4 chrF 与字符级指标
22.5 排序/选择类指标：pass@k 与 Elo
22.6 指标选择决策表
22.7 评测的"元"问题
22.8 常见误区（专题补充）
23.1 知识点映射表
23.2 三条学习路线
23.3 不要重复造的部分
24.1 复习清单（能全部打勾再进下一章）
24.2 自测题（18 题，答案在后）
24.3 答案
25.1 课程与作业（一手）
25.2 讲义 PDF（`slides_w26/`，全部实测 HTTP 200）
25.3 作业与 handout
25.4 教材与公开读物
25.5 关键论文（本笔记引用到的论断）
25.6 已知的不确定项

第 1 章 课程概览：这门课在教什么

1.1 一句话定位

CS224n 教的不是"如何调用一个 LLM API"，而是**如何把"人类语言"写成一个可微的概率模型，并让这个模型在有限数据、有限算力、有限评测口径下真的变好**。整门课可以拆成四层能力：

层	对应讲次	你应当获得的能力
表示层	2–3	把词、子词、句子映射成向量；理解"分布语义"为什么能work、它的边界在哪
建模层	4–6	从 n-gram 到 RNN 到 attention 到 Transformer，每一步解决了前一步的哪个具体缺陷
训练层	7–10	预训练目标怎么选、后训练怎么对齐、参数高效微调怎么省显存、RAG/agent 怎么接外部世界
评测层	11–14	知道 BLEU/困惑度/benchmark/LLM-as-judge 各自测的是什么、会在哪里骗你

课程主页的原话是：2010 年代深度学习用**单个端到端神经网络**取代了任务特定的特征工程；2020 年代**大模型 scaling** 带来了进一步跃迁。所以这门课的主线其实是两条交织的线：

1. **架构线**：one-hot → word2vec → RNN/LSTM → attention → Transformer；
2. **目标线**：任务特定监督 → 语言建模预训练 → 指令微调 → 偏好对齐 → 可验证奖励的 RL。

理解这两条线各自的"为什么"比记住任何单个公式都重要。

1.2 先修（主页明示）

先修	主页口径	在本课里具体用在哪
Python	"Proficiency in Python"，作业全部用 NumPy + PyTorch	A1 的 SVD/可视化、A2 的手写 Embedding/Linear、A3 的 Transformer
微积分 + 线性代数	要求能对多元函数求导、熟悉矩阵向量记号（MATH 51 / CME 100 级别）	反向传播的雅可比、attention 的矩阵形式、LayerNorm 的逐元素缩放
概率统计	高斯、均值、标准差（CS 109 级别）	softmax 的 MLE 解释、困惑度、负采样分布、Bradley-Terry 偏好模型
机器学习基础	CS221/CS229/CS230/CS124 任一；主页说"没有也能上"	损失函数、梯度下降、过拟合与正则化、交叉验证

主页**没有**要求先修深度学习。但本笔记假设你已具备：CS229 的偏差-方差与正则化直觉，以及 CS336 的 Transformer 实现与混合精度训练经验。

1.3 评分与政策（当季原文数字）

项目	权重	细项
Assignments	**48%**	A1 6% + A2 14% + A3 14% + A4 14%

项目	权重	细项
Final Project	**49%**	proposal 8% + milestone 6% + poster 3% + report 32%
Participation	**3%**	4 场客座讲座各 0.375%（合计 1.5%）+ 两次问卷各 0.5% + Ed 贡献/助人分

几条容易被忽略、但会直接扣分的规则：

- **A1 可被丢弃**：如果"不算 A1"能算出更高总分，课程按 A2–A4 各 16% 计。这意味着 A1 是"上手作业"，不是拉分作业。
- **迟交**：每人 6 个 late day，每个作业最多用 3 个；用完后每多迟一天扣总评 1%。项目 proposal/milestone 的 late day **不**在队内共享，final report **可以**共享（需在报告开头声明）。
- **AI 工具**：允许把 ChatGPT/Copilot 当"协作者"讨论，但**直接索取答案或抄袭代码**明确禁止；"用 AI 实质性地完成作业"按 Honor Code 处理。
- **作业不能照抄往年**：主页明确"assignments change from year to year; please do not do assignments from previous years"。
- **评分申诉**：成绩发布后 3 天内在 Gradescope 提交 regrade；助教保留"整份重评"的权利。

1.4 讲次地图（Winter 2026，19 个讲次）

课程共 10 周，周二/周四 16:30–17:50（太平洋时间），地点 NVIDIA Auditorium。下表日期、讲题、幻灯片文件名均从主页日程与 `slides_w26/` 实际下载验证。

#	日期	讲题	讲义文件（`slides_w26/`）	作业节点
1	Jan 6 (Tue)	History of NLP	`lecture01-intro.pdf`、`lecture01-history.pdf`	A1 out
2	Jan 8 (Thu)	Word Vectors	`lecture02-wordvecs.pdf`	
3	Jan 13 (Tue)	Backpropagation & NN Basics	`lecture03-neuralnets.pdf`	A2 out；A1 due
4	Jan 15 (Thu)	Language Models and RNNs	`lecture04-rnnlm.pdf`	
5	Jan 20 (Tue)	Attention and Transformers	`lecture05-transformers.pdf`	
6	Jan 22 (Thu)	Final Projects: Practical Tips	`lecture06-final-project.pdf`	A3 out；A2 due
7	Jan 27 (Tue)	Pretraining (Scaling, Systems, Data)	`lecture07-pretraining.pdf`	
8	Jan 29 (Thu)	Post-training (RLHF, SFT, DPO)	`lecture08-posttraining.pdf`	Proposal + DFP out
9	Feb 3 (Tue)	Efficient Adaptation (Prompting + PEFT)	`lecture09-peft.pdf`	
10	Feb 5 (Thu)	Agents, Tool Use, and RAG	`lecture10-rag-agents.pdf`	A4 out；A3 due

#	日期	讲题	讲义文件（`slides_w26/`）	作业节点
11	Feb 10 (Tue)	Benchmarking and Evaluation	`lecture11-evaluation.pdf`	Proposal due
12	Feb 12 (Thu)	Reasoning 1	`lecture12-reasoning-part1.pdf`	
13	Feb 17 (Tue)	Reasoning 2	`lecture13-reasoning-part2.pdf`	Milestone out
14	Feb 19 (Thu)	Guest: Tokenization & Multilinguality (Julie Kallini)	`lecture14-guest-julie-tokenization-multilinguality.pdf`	A4 due
15	Feb 24 (Tue)	Guest: Interpretability (Been Kim)	**未公开**（见 1.6）	
16	Feb 26 (Thu)	Social and Broader Impacts (Risks)	`lecture16-impact-on-humanity.pdf`	Milestone due
17	Mar 3 (Tue)	Guest: Multimodality (Luke Zettlemoyer)	**未公开**	
18	Mar 5 (Thu)	Guest: Tinker and LoRA Without Regret (John Schulman)	**未公开**	
19	Mar 10 (Tue)	Open Questions in NLP 2026	`lecture19-open-questions.pdf`	
—	Mar 12 (Thu)	No Lecture	—	Final project due
—	Mar 16 (Mon)	Final Project Poster Session 12:15–15:15, AOERC	—	线下学生必须到场

另有 4 个周五的辅助 session（不计入讲次）：Jan 9 Python Review、Jan 16 PyTorch Tutorial、Feb 6 Hugging Face Transformers Tutorial（幻灯片 `materials/hf_transformers_tutorial.pdf`）。

1.5 与 CS336、CS229 的分工

同一篇 Transformer 论文，三个视角看的完全不是一回事。把分工想清楚，能省掉大量重复劳动：

维度	CS224n（本笔记）	CS336（系统视角）	CS229（数学视角）
核心问题	语言任务怎么建模、目标函数怎么设计	模型怎么在 GPU 集群上高效跑起来	估计量/损失函数的统计性质
词表	BPE 的**动机与多语言后果**	BPE **训练算法与工程实现**	—
Attention	为什么它替代 RNN、怎么读 attention map	FlashAttention 的 IO 复杂度、KV cache 显存	核方法/相似度的统计解释
训练	预训练目标、后训练配方、微调策略	并行策略、混合精度、激活重计算、scaling 拟合	梯度下降收敛性、正则化的偏差–方差
评测	BLEU/ROUGE/基准/LLM-judge 的效度	吞吐、延迟、MFU	假设检验、置信区间

维度	CS224n（本笔记）	CS336（系统视角）	CS229（数学视角）
产出	能读懂并复现一篇 ACL 论文的建模部分	能自己写 kernel、跑通多机训练	能推导估计量的性质

一句话：**CS336 让你能把模型训出来，CS224n 让你知道该训什么、以及训出来的东西算什么水平。**

1.6 材料可访问性与本笔记的证据边界

这一节是"事实可追溯"的底盘，后续章节引用具体数字时都以这里列出的文件为准。

材料	可访问性	备注
当季讲义 PDF	✅ 公开直链	17 份，文件名在上表；HTTP 200 实测可下载
课程 notes（readings）	✅ 公开	如 `readings/cs224n_winter2023_lecture1_notes_draft.pdf`、`cs224n-2019-notes05-LM_RNN.pdf` 等，覆盖课程前半段
作业 zip / handout	✅ 公开	`assignments_w26/a1.zip`、`a2.zip` / `a2.pdf`、`a3.pdf`、`a4.pdf`
期末项目 handout	✅ 公开	`project_w25/CS_224n__Default_Final_Project__Build_GPT_2.pdf`、`project/Project_Proposal_Instructions.pdf` 等
起始代码仓库	✅ 公开	DFP 仓库 `github.com/cfifty/public_cs224n_gpt`
当季讲课视频	❌ 需 Canvas 登录	主页明说"not possible to make these videos viewable by non-enrolled students"
2024 完整视频	✅ 公开	YouTube 播放列表（见页首）
L15 Been Kim / L17 Zettlemoyer / L18 Schulman	❌ 当季无公开 PDF	我按 `lecture15/17/18` 的命名规律逐一探测，均返回 404；L15 在 HTML 里链到的是 2024 年 Lambert 的旧讲《Life after DPO》，**不是** Been Kim 的可解释性讲

因此，本笔记对这三讲只做"基于讲题与公开读物的概述"，并明确标注为**推断/概述**，不编造讲者观点与数字。

第 2 章 从 Weaver 到 GPT：NLP 的四次范式转移（L01）

2.1 动机：历史不是装饰，是"假设决定上限"的证据

L01 的学习目标写得很直白：*"understand the paradigm shifts in NLP and how our assumptions about language shape what is possible in each era."* 这一讲由 CS224n 的创建者 Christopher Manning 主讲（客座），把 NLP 分成四个时代。每个时代的"模型"其实是一套关于语言的假设；假设错了，算力再多也没用。

2.2 四个时代与代表性"模型"

时代	年份	核心假设	代表方法	为什么结束
早期探索	1940–1969	语言可以当密码破译；智能=符号操作	机械翻译、McCulloch–Pitts 阈值单元、感知机	ALPAC 报告（1966）承认机器翻译远不成熟；算力不如今天的计算器
符号 NLP	1970–1992	语言=规则系统，意义=逻辑形式	系统文法、LFG、SHRDLU、LUNAR、CHAT-80、Norvig 的推理系统	规则无法覆盖真实语言的歧义与长尾；"AI winter"
统计 NLP	1993–2012	语言=概率分布，标注数据+监督学习	噪声信道模型、HMM、PCFG、CRF、Naive Bayes、MaxEnt、SVM	特征工程到顶；稀疏性/泛化问题
深度学习	2013–	语言=可微表示上的序列预测	word2vec → RNN/seq2seq → attention/Transformer → 预训练/后训练	仍在进行中

2.3 关键事件的"公式化"读法

历史事件之所以值得记，是因为每一个都对应一个至今仍在用的形式化。

(a) 噪声信道（noisy channel）：Shannon 1948 的形式化把"翻译/纠错"写成

$$\hat{y} = \arg\max_{y} P(y)\, P(x \mid y)$$

其中 $P(y)$ 是语言模型（目标语言流利度），$P(x \mid y)$ 是信道模型（目标到源的生成）。CS224n L01 用拼写纠错举例：`acress` 在 `versatile ___ whose` 里，比较 $P(\text{actress} \mid \text{versatile}) \cdot P(\text{whose} \mid \text{actress})$ 与 $P(\text{across} \mid \text{versatile}) \cdot P(\text{whose} \mid \text{across})$，前者约 $817{,}000 \times 10^{-10}$、后者约 239×10^{-10}，于是选 actress。这个例子的教学价值在于：**语言模型从一开始就是"打分器"，不是"生成器"**。

(b) 符号语义组合：`The red apple is on the table` 的经典流水线是 tokenize → parse → 词义查找 → 按规则逐层组合，得到

$$\text{on}\big(\iota(\lambda x.\ \text{apple}(x) \wedge \text{red}(x)),\ \iota(\lambda y.\ \text{table}(y))\big)$$

这条路线的问题是组合爆炸与知识获取瓶颈；但它留下的遗产（句法结构、语义组合性）至今仍是评测的考点。

(c) 分布式表示：Firth 1957 的名言 "You shall know a word by the company it keeps" 把词义定义为上下文分布。2013 年的 word2vec 让这句话第一次可以大规模优化——这就是第 3 章的主题。

2.4 训练细节：数据与算力的共同演化

L01 的历史线里，语言模型那条支线特别值得记，因为它解释了"为什么 2018 年才爆发"：

年份	事件	规模
1913	Markov 用《叶甫盖尼·奥涅金》研究辅音–元音转移概率	字符级
1948	Shannon 用 n-gram 做文本生成与熵的估计	字符/词级
1975	IBM Jelinek 组把"语言模型"定义为 next-token 预测，用于拼写纠错/语音/MT	—
2000	Bengio 等 *A Neural Probabilistic Language Model*	3200 万 token，31K 词表
2007	Brants 等 *Large Language Models in Machine Translation*	2 万亿 token 的 n-gram（最高 5-gram）

年份	事件	规模
2018	GPT / BERT	33 亿 token
2020—	GPT-3、GPT-4、PaLM 2、Llama 3 等	千亿参数、万亿 token

Manning 的总结是三个"不够"：2000 年**算力不够**，2007 年**灵活性不够**，2018 年**数据不够**——三者补齐后才有 2020 年代的 LLM。这句话本身是演讲者的概括（不是定理），但它精确解释了 scaling 的三要素。

2.5 实验现象：深度学习第一次赢在哪

不是 NLP，而是**语音识别**先证明了深度网络的价值。Dahl 等 2012 的对比（L01 引用）：

系统	RT03S FSH WER	Hub5 SWB WER
传统 GMM	27.4	23.6
深度网络（2012）	18.5（−33%）	16.1（−32%）
深度网络（Saon 2017）	8.0（−71%）	5.5（−77%）

机器翻译的转折同样干脆：2014 年第一篇 seq2seq 论文，2016 年 Google Translate 从 SMT 切到 NMT。L01 的原话是"SMT systems, built by hundreds of engineers over many years, outperformed by NMT systems trained by small groups of engineers in a few months"——**这是"端到端"对"流水线"的胜利**，也是本课程方法论的起点。

2.6 常见误区

1. **误区：历史只是"故事"，考试不考。** 实际上 CS224n 后续每一讲都在复用历史里的形式化：噪声信道 → 今天的 reward model + KL 正则；符号组合 → 今天的 compositional generalization 评测；n-gram 稀疏性 → 今天的 smoothing/backoff 直觉。
2. **误区：深度学习"推翻"了符号 NLP。** 更准确的说法是"处理器变了，符号系统仍在"：token 是符号、语法树仍是评测对象、形式语义仍是可解释性研究的工具。
3. **误区：把"参数多"当成唯一解释。** L01 强调的是 data + compute + flexibility 三者同时到位；2007 年那个 2 万亿 token 的 n-gram 模型参数量也不小，但缺灵活性。
4. **误区：以为 MT 的 BLEU 提升是线性的。** 从 SMT 到 NMT 是范式跳跃，不是同一曲线上的外推——这也是为什么"benchmark 分数"必须结合"评测口径"读（第 22 章）。

第 3 章 词向量：从 one-hot 到分布语义（L02）

3.1 动机：one-hot 的两个致命问题

传统 NLP 把词当离散符号：`motel = [0,...,1,...,0]`，维度等于词表大小（常 50 万以上）。这带来两个问题：

1. **无法表达相似性**：`motel` 与 `hotel` 的 one-hot 向量正交，余弦相似度为 0。搜索 "Seattle motel" 匹配不到 "Seattle hotel"。
2. **维度灾难**：每个词一维，任何分类器的参数量都随词表线性膨胀，且每个词独立，无法共享统计强度。

WordNet 之类的同义词集是"人工打补丁"：缺细微差别（"proficient" 只是某些语境下等于 "good"）、缺新词（"wicked/badass/nifty"）、带主观偏见、且无法计算细粒度相似度。

分布语义（distributional semantics） 是替代方案：一个词的意义由它经常出现的上下文决定。于是我们把词映射到低维稠密向量，让"上下文相似"变成"向量相似"（点积大）。

3.2 模型公式：skip-gram 与 softmax

给定语料 $w_1, \dots, w_T$、窗口大小 m，skip-gram 用中心词预测窗口内的上下文词。似然与目标函数：

$$L(\theta) = \prod_{t=1}^{T} \prod_{\substack{-m \le j \le m \\ j \neq 0}} P(w_{t+j} \mid w_t; \theta), \qquad J(\theta) = -\frac{1}{T} \log L(\theta)$$

每个词学**两个**向量：v_w（作中心词）与 u_w（作上下文词）。对中心词 c、上下文词 o：

$$P(o \mid c) = \frac{\exp(u_o^\top v_c)}{\sum_{w \in V} \exp(u_w^\top v_c)}$$

这个式子的三个部件各有含义：点积 $u_o^\top v_c$ 度量相似度；指数保证正数；分母把整个词表归一化成概率分布。合起来就是 softmax $\mathbb{R}^n \to (0,1)^n$："max" 是因为放大最大值，"soft" 是因为仍给其他项留概率。

为什么两个向量？ 因为 $u^\top v$ 对 (u, v) 对称，若只用一个向量，w 与 w' 的相似度会和自身平方混在一起，优化更难。工程上训练完通常把两个向量**相加或平均**作为最终词向量。

训练：SGD/小批量，每步只更新窗口内出现的词向量，所以梯度**极其稀疏**——这也是负采样能把训练加速几个数量级的原因。

3.3 训练细节：负采样与高频词下采样

朴素 softmax 的分母要遍历整个词表（$|V|$ 常 10 万–100 万），不可接受。**负采样（negative sampling）** 把"多分类"改成"二分类"：

$$J_{\text{neg-sample}}(u_o, v_c, U) = -\log \sigma(u_o^\top v_c) - \sum_{k \in K} \log \sigma(-u_k^\top v_c)$$

其中 K 个负样本按 $P(w) = U(w)^{3/4} / Z$ 采样（$U(w)$ 是 unigram 频率）。3/4 次方的作用是把分布"压平"：高频词被降权、低频词被提权，缓解"负样本几乎总是 the/of"的问题。

配套的两个细节：

- **高频词下采样（subsampling）**：以与频率相关的概率丢弃高频词（如 "the"），既加速又改善低频词的表示。
- **动态窗口**：窗口大小在 $[1, m]$ 内随机，离中心越近的词越容易被纳入，等价于给近距离词更高权重。

3.4 实验现象：类比、相似度与"线性结构"

词向量最出名的现象是**类比**：`man : woman :: king : ?` 用 $v_{\text{king}} - v_{\text{man}} + v_{\text{woman}}$ 的最近邻回答，常得到 queen。评估时**要把输入词从候选里剔除**，否则最近邻永远是输入自己。

两类内在（intrinsic）评估：

评估	做法	报告指标	已知问题
类比	语义/句法类比题集，取 top-1 准确率	准确率	信息可能存在但非线性；剔除输入词后仍受词频影响
相似度	与人类相似度评分（如 WordSim353）比较	Spearman 相关	人类评分本身噪声大、粒度粗

L02 给的 WordSim353 片段很能说明"人类评分"的粗糙：`tiger-cat` 7.35，`tiger-tiger` 10，`book-paper` 7.46，`stock-jaguar` 0.92。注意 `tiger-tiger` 是 10 分——词表里同一个词的自相似度，会被算进相关系数，抬高指标。

外在（extrinsic）评估则是把词向量塞进 NER/分类器，看下游指标。主页的口径很清楚：内在评估快、可解释，但"unless correlation to real task is established"就不知道有没有用；外在评估慢、归因难，但"replacing exactly one subsystem with another improves accuracy → Winning"。

3.5 常见误区

1. **误区：余弦相似度高 = 语义相同。** 反义词（happy/sad）在分布上高度相似，余弦距离可能比同义词更近——A1 的 Q2.3 就是专门考这个反例。
2. **误区：类比能力是"推理"。** 它更像是词向量空间里近似的平移不变性；对多义词、低频词、形态丰富的语言经常失效（`hand:glove::foot:?` 在 GloVe 上不会给出 sock）。
3. **误区：窗口越大越好。** 小窗口偏向句法相似（可替换性），大窗口偏向语义/主题相似；这是**任务相关**的超参，不是单调改进。
4. **误区：词向量是"一词一义"。** 静态词向量把多义词压成一个点；`record`（名词/动词）在 GloVe 里是同一个向量。这正是第 8 章 ELMo/BERT 的动机。
5. **误区：训练完的向量可以直接比较不同模型。** 不同维数、不同语料、不同归一化的向量空间不可比；跨模型比较必须先对齐（Procrustes）或只在同一模型内比较。

第 4 章 反向传播与神经网络基础（L03）

4.1 动机：从"线性分类器"到"可学习的表示"

L03 用 **NER（named entity recognition，命名实体识别）** 做贯穿例子：`Last night , Paris Hilton wowed in a sequin gown .` 要给每个词打 PER/LOC/ORG/O 标签。窗口分类器把中心词前后各 2 个词的词向量拼起来：

$$x_{\text{window}} = [x_{\text{museums}}; x_{\text{in}}; x_{\text{Paris}}; x_{\text{are}}; x_{\text{amazing}}] \in \mathbb{R}^{5d}$$

线性分类器 $s = Wx + b$ 的问题是决策边界是线性的；神经网络的关键差别不是"层数多"，而是**同时学习 W 和词表示 x**：词向量被"移动"到一个更容易线性分类的中间空间。概念上这是一个 embedding layer：$x = Le$。

4.2 模型公式：从 score 到 cross-entropy

单隐层分类器：

$$h = f(Wx + b_1), \qquad s = Uh + b_2, \qquad \hat{y} = \text{softmax}(s)$$

f 是逐元素非线性。**没有非线性，多层等价于一层**：$W_2(W_1x) = (W_2W_1)x$。所以非线性不是"装饰"，而是深度模型表达力的来源。

常用非线性（L03 列出，含 Transformer 时代的变体）：

名称	公式	特点/用途
logistic / sigmoid	$\sigma(z) = 1/(1+e^{-z})$	输出概率；深层网络易饱和
tanh	$\tanh(z) = 2\sigma(2z) - 1$	零中心，比 sigmoid 好训
ReLU	$\max(z, 0)$	梯度回传好、训练快；有 dead zone
Leaky/Parametric ReLU	$\max(\alpha z, z)$	缓解 dead ReLU
GELU	$x \cdot P(X \le x),\ X \sim N(0,1) \approx x \cdot \sigma(1.702x)$	Transformer 常用
SiLU / Swish	$x \cdot \sigma(x)$ / $x \cdot \sigma(\beta x)$	平滑版 ReLU
GLU / SwiGLU	$(xV + v) \otimes \sigma(xW + b)$	Llama3、Qwen3 等 FFN 常用

损失用**交叉熵（cross-entropy）**：设真实分布 p 是 one-hot，模型分布 $q = \hat{y}$，则

$$H(p, q) = -\sum_i p_i \log q_i = -\log \hat{y}_{y_i}$$

因为 one-hot 只剩真实类那一项。这也解释了"最小化负对数似然 = 最小化交叉熵"。

4.3 训练细节：雅可比、shape convention 与计算图

雅可比（Jacobian） 是 m 输出对 n 输入的 $m \times n$ 偏导矩阵；链式法则在多元情形就是雅可比相乘。逐元素激活的雅可比是对角矩阵，这是反向传播高效的关键之一。

课程反复强调 **shape convention（形状约定）**：参数的梯度形状 = 参数本身的形状。纯数学的雅可比形式便于链式推导，但实现 SGD 时若形状不匹配会非常痛苦。两种做法：

1. 用雅可比推导，最后转置/重排成参数形状；
2. 全程按形状约定，靠维度检查决定何时转置。

以 $s = Wx + b$ 为例，若上游梯度是 $\delta = \partial s / \partial z$（与 z 同形），则

$$\frac{\partial s}{\partial W} = \delta x^\top, \qquad \frac{\partial s}{\partial b} = \delta, \qquad \frac{\partial s}{\partial x} = W^\top \delta$$

$\delta x^\top$ 是外积：每个输入分量与每个输出误差相乘，正好是 W 的形状。

反向传播的本质是"链式法则 + 复用中间结果"：

```
前向：按拓扑序计算每个节点的值，保存中间量
反向：初始化输出梯度 = 1
      按逆拓扑序，对每个节点：
          downstream_grad = upstream_grad · local_jacobian
      同一节点被多条路径使用时，梯度相加
```

四条节点直觉（L03 的原话）：

节点	行为
`+`	**分发**上游梯度给每个加数（梯度原样传回）
`max`	**路由**上游梯度到最大值那一路，其余为 0
`*`	**交换**：对 x 的梯度 = 上游梯度 × y，对 y 的梯度 = 上游梯度 × x
分支汇合	梯度**相加**

4.4 实验现象：为什么手推梯度仍然值得学

现代框架（PyTorch/TensorFlow）用自动微分（automatic differentiation）替你算梯度，但 L03 给了三条"仍然要学"的理由：

1. **反向传播不是永远 work**：梯度消失/爆炸、dead ReLU、NaN 都来自梯度层面的问题；
2. **调试需要看梯度**：梯度范数、每层梯度均值、梯度与数值梯度的对比；
3. **自定义层/损失时需要写局部导数**：框架只负责把局部导数串起来。

数值梯度检查（gradient checking）：

$$\frac{\partial f}{\partial \theta_i} \approx \frac{f(\theta_i + h) - f(\theta_i - h)}{2h}, \qquad h \approx 10^{-4}$$

中心差分比前向差分精度高一个量级；但它要对每个参数重算 f，非常慢，只适合小模型/单层抽查。

4.5 常见误区

1. **误区：`loss.backward()` 之后不用清零梯度。** PyTorch 默认**累加**梯度，忘记 `optimizer.zero_grad()` 会让梯度跨 batch 累积，表现为"训练突然爆炸"或"学习率等效变大"。
2. **误区：形状对了就没错。** 广播（broadcasting）会让错误的形状悄悄"对上"。A2/A3 都建议加 `assert` 检查形状。
3. **误区：ReLU 一定比 sigmoid 好。** 在需要概率输出或小网络上 sigmoid/tanh 仍常用；ReLU 的收益主要在深层网络的梯度回传。
4. **误区：梯度检查通过 = 实现正确。** 数值梯度只能验证"这一点附近"的导数；mask、in-place 操作、dropout 的随机性都会让检查失真（检查时要关 dropout、固定种子）。
5. **误区：把 softmax 和交叉熵分开实现。** 分开写容易在数值上溢出（`exp` 过大）；应使用 `log_softmax` + `nll_loss` 的稳定组合。

第 5 章 语言模型与 RNN（L04）

5.1 动机：一切生成任务的底座

语言模型（language model, LM） 的任务是：给定词序列，预测下一个词的概率分布。形式上

$$P(w_{t+1} \mid w_1, \ldots, w_t), \qquad \text{或整体 } P(w_1, \ldots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_{<t})$$

L04 的原话是"This is the most important concept in the class! Leads to most of modern NLP"。原因很直接：预测下一个词需要语法（`I put ___ fork down`）、指代（`over ___ shoulder`）、主题（`fish, turtles, seals, and ___`）、情感（`The movie was ___`）、甚至一点推理与算术（斐波那契数列）。ChatGPT 本质上就是一个 LM。

5.2 模型公式：n-gram 与马尔可夫假设

n-gram 模型用**马尔可夫假设**截断历史：

$$P(w_t \mid w_1, \ldots, w_{t-1}) \approx P(w_t \mid w_{t-n+1}, \ldots, w_{t-1}) = \frac{C(w_{t-n+1}, \ldots, w_t)}{C(w_{t-n+1}, \ldots, w_{t-1})}$$

例如 4-gram 下 `P(books | students opened their) = C(students opened their books)/C(students opened their) = 400/1000 = 0.4`。

两个稀疏性问题及部分解法：

问题	现象	解法
分子为 0	某个 n-gram 没出现 → 概率 0	平滑（smoothing），如加 δ、Kneser-Ney
分母为 0	历史没出现 → 无法计算任何词	回退（backoff）到更短的历史

n-gram 的存储随 n 与语料线性增长，实际 n 很少超过 5。用 4-gram 生成的文本"surprisingly grammatical... but incoherent"——**局部语法对、全局语义散**，这是"需要更长上下文"的直观证据。

5.3 模型公式：从固定窗口到 RNN

固定窗口神经 LM（Bengio 等 2000/2003）：把前 $n-1$ 个词的词向量拼接，过隐层，输出 softmax。优点是**没有稀疏性问题**、不需要存 n-gram；缺点是窗口固定、W 随窗口变大、且不同位置用完全不同的权重（没有位置对称性）。

RNN（recurrent neural network） 的核心是"同一组权重反复使用"：

$$h_t = \tanh\left(W_{hh} h_{t-1} + W_{xh} x_t + b_h\right), \qquad \hat{y}_t = \mathrm{softmax}\left(W_{hy} h_t + b_y\right)$$

h_0 是初始隐状态（可学习或置零）。优点：能处理任意长度、模型大小不随上下文增长、位置对称；缺点：**循环计算无法并行**，且实践中很难用到很多步之前的信息（梯度问题）。

训练：每个时间步的损失是交叉熵

$$J_t = -\log \hat{y}_t[w_{t+1}], \qquad J = \frac{1}{T}\sum_t J_t$$

训练时把**真实**的上一词喂给下一步（teacher forcing），而不是模型自己的预测；推理时才把自己的输出喂回去。这个 train/test 差异叫 **exposure bias**，是 seq2seq 时代的老问题（第 14 章的同策略蒸馏正是针对它）。

BPTT（backpropagation through time）：对重复使用的 W_{hh}，梯度是各时间步梯度之和："the gradient w.r.t. a repeated weight is the sum of the gradient w.r.t. each time it appears"。实践上常**截断**到约 20 步（truncated BPTT），用状态延续来近似长程依赖。

困惑度（perplexity, PPL） 是 LM 的标准指标：

$$\mathrm{PPL} = \exp\left(-\frac{1}{T}\sum_{t=1}^{T} \log P(w_t \mid w_{<t})\right) = \exp(\text{交叉熵})$$

直观理解：PPL 等于"模型在每一步平均需要在多少个词里犹豫"。PPL 越低越好，但**不同词表/分词下的 PPL 不可直接比较**（第 22 章会展开）。

5.4 训练细节：梯度消失/爆炸与 LSTM

对 $h_t = \tanh(W h_{t-1} + \dots)$ 反向传播，梯度里会出现连乘

$$\frac{\partial h_t}{\partial h_{t-k}} = \prod_{i=0}^{k-1} \frac{\partial h_{t-i}}{\partial h_{t-i-1}}$$

若这些雅可比的谱范数 < 1，梯度随 k 指数衰减（vanishing）；若 > 1，指数放大（exploding）。

问题	后果	解法
梯度消失	只能学到近处效应，长距离依赖（如"tickets"与末尾的空格）学不到	LSTM/GRU 的门控 + 加法记忆；attention/residual 提供"直连"
梯度爆炸	更新步过大，loss 变 NaN，训练崩	**梯度裁剪（gradient clipping）**：若 $\|\hat{g}\| > \tau$，令 $g \leftarrow \tau \hat{g} / \|\hat{g}\|$

裁剪的直觉是"方向不变、步长变小"。L04 的评价是：爆炸梯度是"easy problem to solve"，消失梯度才是结构性问题。

LSTM（long short-term memory） 的关键是引入一条**加法**的 cell state 通道，让梯度可以"线性直通"：

$$\begin{aligned}
i_t &= \sigma(W_i [h_{t-1}; x_t] + b_i) && \text{(input gate)} \\
f_t &= \sigma(W_f [h_{t-1}; x_t] + b_f) && \text{(forget gate)} \\
o_t &= \sigma(W_o [h_{t-1}; x_t] + b_o) && \text{(output gate)} \\
g_t &= \tanh(W_g [h_{t-1}; x_t] + b_g) && \text{(candidate)} \\
c_t &= f_t \odot c_{t-1} + i_t \odot g_t && \text{(cell state)} \\
h_t &= o_t \odot \tanh(c_t)
\end{aligned}$$

c_t 的更新是"遗忘 + 写入"，$\partial c_t / \partial c_{t-1} = f_t$ 是对角矩阵：只要 f_t 接近 1，梯度就能几乎无损地传很远。这就是"separate memory which is added to"的含义。

5.5 实验现象：从 LM 到 seq2seq 与瓶颈

seq2seq（sequence-to-sequence） 把 encoder 的最后一个隐状态当作源句的"编码"，喂给 decoder：

$$P(y_1, \dots, y_T \mid x) = \prod_t P(y_t \mid y_{<t}, h_T^{\text{enc}})$$

它是**条件语言模型（conditional LM）**：decoder 是 LM，但条件于源句 x。训练时对整句求和的负对数似然端到端反传。

问题是**瓶颈（bottleneck）**：源句所有信息必须压进一个固定维度向量，句子越长丢得越多。L04 末尾把这个问题留给了 attention（下一讲）："on each step of the decoder, use direct connection to the encoder to focus on a particular part of the source sequence"。

5.6 常见误区

1. **误区：PPL 越低模型越"好"。** PPL 只衡量对**同一种分词、同一测试集**的拟合；词表越小 PPL 越容易低。跨 tokenizer 比较 PPL 是无意义的。

2. **误区：teacher forcing 只是"训练技巧"。** 它改变了训练分布与推理分布，是 exposure bias 的根源；后面 DPO/同策略蒸馏都在处理这个"训练–推理不一致"。
3. **误区：LSTM 解决了梯度消失。** 它**缓解**了消失（让 $f_t \approx 1$ 时梯度可直通），但没有消除；真正的"直连"来自 attention/residual。
4. **误区：截断 BPTT 只是省显存。** 它还改变了优化目标（只看 20 步的依赖），是"模型能力"与"计算预算"的折中。
5. **误区：n-gram 已经过时。** 它的平滑/回退思想（Kneser–Ney、Good–Turing）在今天的校准与稀有事件建模里仍然出现（第 16 章的 Good–Turing 就是同一思想）。

第 6 章 注意力机制与 Transformer（L05）

6.1 动机：瓶颈与"直连"

seq2seq 的瓶颈是"所有源信息压进一个向量"。attention 的解法是：decoder 每一步都**直接看** encoder 的所有隐状态，按相关性加权求和。直觉上它把"查表"软化了：查表是 query 精确匹配一个 key 取 value；attention 是 query 与所有 key 软匹配，得到权重 0–1，再对 value 加权求和。

L05 的递进很清楚：先用 mean-pooling 说明"平均"是一种最笨的信息传递，再把"平均"换成"学出来的加权平均"。

6.2 模型公式：从 cross-attention 到 self-attention

cross-attention（seq2seq + attention）：设 encoder 隐状态 $h_1, \dots, h_N$，decoder 当前状态 s_t，

$$e_{t,i} = s_t^\top h_i, \qquad \alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_j \exp(e_{t,j})}, \qquad c_t = \sum_i \alpha_{t,i} h_i$$

然后把 $[s_t; c_t]$ 送进输出层。三种打分函数都出现过：点积 $s^\top h$、双线性 $s^\top W h$、加性 $v^\top \tanh(W_1 s + W_2 h)$。

self-attention：Q、K、V 都来自同一序列。对每个词向量 x_i：

$$q_i = Q x_i, \quad k_i = K x_i, \quad v_i = V x_i, \qquad e_{ij} = q_i^\top k_j, \qquad \alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{j'} \exp(e_{ij'})}, \qquad o_i = \sum_j \alpha_{ij} v_j$$

矩阵形式（$X \in \mathbb{R}^{n \times d}$）：

$$\mathrm{Attention}(Q, K, V) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

为什么要除以 $\sqrt{d_k}$？ 当维度变大时点积的方差随 d_k 增长，softmax 输入过大会进入饱和区、梯度趋零。缩放让 logits 的尺度与维度无关。

多头（multi-head）：让模型在不同子空间、不同位置同时"看"：

$$\mathrm{head}_\ell = \mathrm{Attention}(XQ_\ell, XK_\ell, XV_\ell), \qquad \mathrm{MultiHead} = \mathrm{Concat}(\mathrm{head}_1, \dots, \mathrm{head}_h) W^O$$

每个头的维度是 d/h，所以总计算量与单头同维基本相同——"多头不是更贵，而是把同样的预算切成 h 份"。

6.3 训练细节：位置、非线性、mask、残差与 LayerNorm

self-attention 本身是**置换等变**的：打乱输入顺序，输出只是跟着打乱。要处理语言，必须补四件东西。

缺失	补法	细节
顺序信息	**位置表示（position representation）**	正弦/余弦固定编码，或可学习绝对位置；加在输入上
非线性	**FFN（position-wise feed-forward）**	$m_i = W_2\,\mathrm{ReLU}(W_1 o_i + b_1) + b_2$，逐位置共享
不能看未来	**masking**	令 $e_{ij} = -\infty$（$j > i$），softmax 后未来权重为 0
深层难训	**残差 + LayerNorm**	$X^{(i)} = X^{(i-1)} + \mathrm{Layer}(X^{(i-1)})$；LN 在特征维归一化

正弦位置编码：

$$p_{i,2j} = \sin\left(\frac{i}{10000^{2j/d}}\right), \qquad p_{i,2j+1} = \cos\left(\frac{i}{10000^{2j/d}}\right)$$

优点是不用学、理论上可外推到更长序列；缺点是"extrapolation doesn't really work"，且不可学习。可学习绝对位置更灵活，但完全无法外推。这也是后来 RoPE 的动机（第 20 章）。

LayerNorm：对单个词向量 $x \in \mathbb{R}^d$，

$$\mu = \frac{1}{d}\sum_j x_j, \quad \sigma^2 = \frac{1}{d}\sum_j (x_j - \mu)^2, \quad \mathrm{LN}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \gamma + \beta$$

与 BatchNorm 的差别：LN 在**特征维**统计，与 batch 大小无关，因此适合变长序列和小 batch。

三种堆叠方式：

结构	mask	用途
Transformer Encoder	无 mask（双向）	理解类任务、BERT 类预训练
Transformer Decoder	有 causal mask	生成、GPT 类 LM
Encoder-Decoder	encoder 双向 + decoder causal + cross-attention	翻译、T5、Whisper

cross-attention 的细节：$k_i = Kh_i$, $v_i = Vh_i$ 来自 encoder（像 memory），$q_i = Qz_i$ 来自 decoder。

6.4 实验现象：更快、更好，但二次代价

原论文（Vaswani 等 2017）在 WMT 2014 英德/英法上同时取得更好的 BLEU 与更低的训练成本——L05 的措辞是"Not just better Machine Translation BLEU scores. Also more efficient to train!"。这打破了"RNN 的顺序归纳偏置是必要的"这一直觉。

但 Transformer 留下两个待解问题（L05 末尾自己提出）：

1. **二次复杂度**：$QK^\top$ 是 $n \times n$，$n = 50{,}000$ 时长文档不可行；
2. **位置表示**：绝对索引是不是最好的？

L05 还给了一个反直觉的观察：模型越大，self-attention 之外的算力占比越高，所以**在实践中几乎没人真的把二次注意力换掉**——"the cheaper methods tend not to work as well at scale"。这句话是理解后续 long-context 研究（第 14 章）的关键前提。

6.5 常见误区

1. **误区：attention 权重就是"解释"。** attention map 只是某层某头的一次加权，多个头、多层、残差都会改变信息流；把它当因果解释会被 attention-is-not-explanation 系列工作反驳。
2. **误区：mask 是"把未来位置删掉"。** 实现上是把 logits 设为 $-\infty$（或极大负数），softmax 后为 0；如果整行都被 mask（padding 行的极端情况），会出现 NaN，需要小心处理。
3. **误区：位置编码加在每一层。** 标准做法只在**第一层输入**加一次；每层都加会重复注入位置信息。
4. **误区：多头一定比单头好。** 多头的收益来自"不同子空间"，但头数过多会让每头维度太小；而且头之间有冗余（可剪枝）。
5. **误区：Post-LN 与 Pre-LN 等价。** 原论文是 Post-LN（LN 在残差之后），深层需要 warmup；Pre-LN 更稳定、是现代大模型默认，但表达细节不同（第 20 章）。

第 7 章 期末项目与实验方法论（L06）

7.1 动机：从"做作业"到"做研究"

L06 不是模型课，而是整门课唯一讲**研究流程**的一讲。它的价值在于：期末项目占 49%，而大多数失分不是"模型不够新"，而是"数据/评测/基线没设计好"。这一讲把"如何选题、如何找数据、如何调模型、如何写报告"压缩成 55 页。

7.2 模型公式：项目的两种形态

形态	内容	适合谁	评分重心
Default Final Project（DFP）	实现 GPT-2 的 attention/block/embedding，加载 HuggingFace 权重，微调做 SST/CFIMDB 情感分类，再扩展到 Quora 复述检测（cloze 形式）与莎士比亚十四行诗生成（chrF 评测）	没有明确选题、想要明确目标和 leaderboard 的人	实现正确性 + 扩展实验的深度
Custom Final Project	自选"人类语言 + 神经网络"的问题，需提交 proposal 并获批准，配 mentor	已有研究兴趣、想自己定义问题的人	问题定义 + 实验设计 + 分析

L06 的原话很值得记：DFP "is more guided, but it should be the same amount of work"；两条路的区别是"是否要自己找问题"，不是"难度高低"。

7.3 训练细节：实验方法论（这一节比任何模型都重要）

讲义给了一套非常具体的调试流程，我把它整理成可执行清单：

1. **先在小数据上过拟合**：取 4–8 个样本（或合成数据），训练到 **100% 训练准确率**。做不到，说明模型容量不够或代码有 bug，不要继续往下调。
2. **再上全量数据**：仍应接近 100% 训练准确率；如果差很远，"you probably want to consider a more powerful model"。

3. **然后才治过拟合**：L2、dropout（讲义原话："generous dropout is the secret to success"）、早停。
4. **数据划分纪律**：train / tune（调超参）/ dev（看进度）/ dev2（防止对 dev 过拟合）/ test（只在最后用一次）。讲义强调这些集合必须**完全不相交**，"it is invalid to give results testing on material you have trained on"。
5. **增量式加组件**：先跑通最简单的模型，再加 attention、再加预训练、再加数据增强；每加一个都要能说清"它带来了多少提升"。
6. **错误分析**：看数据分布、看模型输出、按错误类型分类计数（不是只看一个总指标）。

7.4 实验现象：什么样的项目拿高分

讲义对"项目类型"给了五类：应用/任务型、复现复杂架构、提出新模型/变体、分析型（模型学到了什么语言知识）、理论/语言学型。共同要求是"experiments with numbers and ablations"。

评分口径（L06 原文要点）：

- 报告质量"very important to your grade"；
- 允许用现成代码/库/论文，但必须引用并说明哪部分是你的贡献；
- 团队 1–3 人，人多则项目要相应更大；
- proposal 8%（形成性评价，主要看"数据是否拿得到、评测是否可做、有没有基线"）、milestone 6%（应完成一半以上并有初步结果）、poster 3%、report 32%。

7.5 常见误区

1. **误区：先搭最复杂的模型。** 讲义明确反对："It's hard to fix a complex but broken model."
2. **误区：在 test 上反复调。** 每看一次 test 就多一分过拟合；只有一次机会。
3. **误区：用 ChatGPT 的输出当自己的贡献。** 讲义原话：评分看"what you have done"，不是"look, it works zero shot"。
4. **误区：数据越大越好。** 自采数据"tricky to do fast"，很容易把整个项目时间耗在标注上。
5. **误区：只报最好的一组数。** 没有消融、没有方差/多种子，结果不可信。

第 8 章 预训练：子词、三种架构与扩展（L07）

8.1 动机：从"预训练词向量"到"预训练整个模型"

2017 年的做法是：加载预训练词向量（无上下文），其余参数随机初始化，在任务数据上训练。问题是**下游数据不够教语言**，且"most of the parameters in our network are randomly initialized"。

现代做法是：**几乎全部参数都由预训练初始化**。预训练的方式是"遮住输入的一部分，让模型重建"——这个"填空"任务迫使模型学习语法、指代、主题、情感乃至推理。L07 的原话："Pretraining methods hide parts of the input from the model, and train the model to reconstruct those parts."

8.2 模型公式：三种架构 × 三种目标

先解决词表：固定词表 + UNK 会把拼写错误、新词、形态变化全部丢掉。**子词（subword）** 方案用 BPE 之类的算法学"词的片段"：

```
BPE 训练：
1. 词表初始化为所有字符（+ 词尾符号）
2. 统计语料中相邻符号对的频次
3. 合并频次最高的对，加入词表
4. 重复 2–3 直到达到目标词表大小
```

编码新词时按**合并顺序**依次应用规则。于是 `taaaaasty → taa## aaa## sty`、`laern → la## ern##`，罕见词被拆成可组合的片段，**没有 OOV**。

三种架构对应三种预训练目标：

架构	目标	代表	能看未来?	天然用途
Encoder	**Masked LM**：遮 15% 的（子）词，重建	BERT、RoBERTa、SpanBERT	✅ 双向	理解/分类/抽取
Encoder–Decoder	**Span corruption**：挖掉不同长度 span，用哨兵占位，decoder 重建	T5、mT5	encoder 双向 / decoder 单向	条件生成、翻译、QA
Decoder	**Causal LM**：预测下一个 token	GPT 系列、Llama	❌ 单向	生成、few-shot、对话

BERT 的 Masked LM 细节（L07 原文）：随机选 15% 的 token；其中 80% 换成 `[MASK]`、10% 换成随机 token、10% 保持不变（但仍要预测）。后两个 10% 是为了"不让模型偷懒"——如果被遮位置永远是 `[MASK]`，模型只需学 `[MASK]` 的表示，不必学其他词的表示。微调时**没有** `[MASK]`，这个训练/推理不一致是 BERT 的著名设计缺陷。

BERT 还有 **NSP（next sentence prediction）**：输入两个连续文本块，预测第二块是否紧跟第一块。后来的工作（RoBERTa）证明它**不必要**。

8.3 训练细节：规模、算力与"预训练一次，微调多次"

模型	层数	隐维	头数	参数	数据	算力
BERT–base	12	768	12	110M	BooksCorpus 800M 词 + 英文 Wikipedia 2,500M 词	64 块 TPU，4 天
BERT–large	24	1024	16	340M	同上	—
GPT	12	768	12	117M	BooksCorpus 7000+ 本书	—
GPT–2	48（最大版）	1600	25	1.5B	WebText 40GB	—
GPT–3	96	12288	96	175B	300B token	—

L07 的结论是："Pretraining is expensive and impractical on a single GPU. Finetuning is practical and common on a single GPU." 这就是"**pretrain once, finetune many times**"的经济学。

扩展（scaling）部分给了两个至今仍被引用的判断：

- 规模化能可靠降低困惑度，且可以**预测**（scaling laws）；

- GPT-3 的 175B 参数 / 300B token 配比**不是**最优的；训练成本大致 $\propto$ 参数 × token，后来有 70B 模型在更优配比下超过更大模型。

8.4 实验现象：预训练到底教会了什么

L07 用一组填空例子说明预训练学到的东西：

例子	需要的知识
`Stanford University is located in ___, California.`	事实（trivia）
`I put ___ fork down on the table.`	句法（冠词）
`... checking for traffic over ___ shoulder.`	指代（coreference）
`... fish, turtles, seals, and ___.`	词汇语义/主题
`The movie was ___.`	情感
`Zuko left the ___.`	常识推理（较难）
`1,1,2,3,5,8,13,21,___`	一点算术（模型并未真正学会斐波那契）

同时它也学到**不该学的**：种族/性别偏见、训练数据中的隐私信息（membership inference 可以恢复部分训练数据）、以及受版权保护的内容。这不是"副作用"，而是"用互联网数据做最大似然"的必然结果。

8.5 常见误区

1. **误区：BERT 的 `[MASK]` 只是数据增强。** 它改变了目标函数（去噪自编码），也造成了预训练/微调不一致；后来的 SpanBERT、Whole Word Masking 都在修这个问题。
2. **误区：Encoder 比 Decoder 强。** 两者只是"能不能看未来"的取舍：分类/NER 用 encoder 更自然，生成必须用 decoder（或 encoder-decoder）。
3. **误区：NSP 是 BERT 成功的关键。** RoBERTa 去掉 NSP、训练更久、数据更多，效果更好——"more compute, more data can improve pretraining even when not changing the underlying Transformer"。
4. **误区：子词是"更小的词"。** 子词边界是**统计**的，常与语言学边界不符（`Milky`、`looked` 都会被切开），这直接导致拼写类任务变难（第 15 章）。
5. **误区：预训练数据越大越好。** 数据质量、去重、配比、以及"是否与评测集重叠"同样关键（第 12、22 章）。

第 9 章 后训练：SFT、RLHF 与 DPO（L08）

9.1 动机：语言建模 ≠ 助手

预训练模型只会"接话"：给它 `Human: ...`，它可能续写出更多 `Human:`。L08 用一张图说明"not aligned with user intent"：模型可能给出冗长、回避、或与指令无关的回答。

指令微调（instruction fine-tuning） 的第一步很朴素：收集 `(instruction, output)` 对，监督微调。但两个更深的问题浮现：

1. 开放式生成（写故事）**没有唯一正确答案**；
2. token 级似然把所有错误等同看待，而"some errors are worse than others"。

于是需要直接优化"人类偏好"。

9.2 模型公式：从奖励到 DPO

奖励建模（reward modeling）：人类对两个回答做**成对比较**（pairwise comparison）比打绝对分更可靠。用 Bradley–Terry 模型：

$$P(y_w \succ y_l \mid x) = \sigma\big(RM_\phi(x, y_w) - RM_\phi(x, y_l)\big), \qquad J_{RM}(\phi) = -\mathbb{E}_{(x,y_w,y_l)\sim D} \log \sigma\big(RM_\phi(x, y_w) - RM_\phi(x, y_l)\big)$$

RLHF 的目标：最大化奖励，同时不要偏离参考模型太远（KL 正则）：

$$\max_\theta \ \mathbb{E}_{\hat{y}\sim p_\theta(\cdot|x)} \left[RM_\phi(x, \hat{y}) - \beta \log \frac{p_\theta(\hat{y} \mid x)}{p_{PT}(\hat{y} \mid x)} \right]$$

KL 项的作用是"防止 reward hacking"：模型可以找到奖励模型的高分漏洞，KL 把它拉回预训练分布。

DPO 的推导（L08 的核心）：

上述目标有闭式最优解

$$p^*(\hat{y} \mid x) = \frac{1}{Z(x)} p_{PT}(\hat{y} \mid x) \exp\left(\frac{1}{\beta} RM(x, \hat{y})\right)$$

反解出奖励：

$$RM(x, \hat{y}) = \beta \log \frac{p^*(\hat{y} \mid x)}{p_{PT}(\hat{y} \mid x)} + \beta \log Z(x)$$

把它代回 Bradley–Terry，$\log Z(x)$ 在"两个奖励相减"时**抵消**，得到可直接用偏好数据优化的损失：

$$J_{DPO}(\theta) = -\mathbb{E}_{(x,y_w,y_l)\sim D} \log \sigma\left(\beta \log \frac{p_\theta(y_w \mid x)}{p_{PT}(y_w \mid x)} - \beta \log \frac{p_\theta(y_l \mid x)}{p_{PT}(y_l \mid x)}\right)$$

这就是"把 RL 换成一个加权的最大似然"：不需要 reward model、不需要在线采样、不需要 value network。L08 的结论是"open source LLMs now almost all just use DPO (and it works well!)"。

GRPO 是另一条线：去掉 value network，用**同一 prompt 的一组回答的组内标准化奖励**作为 advantage：

$$\hat{A}_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G)}$$

这直接省掉了 PPO 的 critic（第 13 章）。

9.3 训练细节：三阶段流水线与数据来源

标准 RLHF 流水线（InstructGPT/ChatGPT 的口径）：

阶段	数据	目标	备注
1. SFT	人工写的示范（demonstration）	监督下一个 token	让模型学会"格式"
2. RM	成对比较	Bradley–Terry	先验证 RM 在留出人类判断上的准确率

阶段	数据	目标	备注
3. RL	prompt + RM	PPO + KL	在线采样，成本高、超参敏感

L08 给的规模化数字：InstructGPT 覆盖"30k tasks"；FLAN-T5 在 1.8K 任务上指令微调；Super-NaturalInstructions 有 1.6K+ 任务、3M+ 示例；Alpaca 用 52K 指令数据微调 LLaMA-7B；LIMA 说明"少量高质量样本"也能有不错效果。

数据来源的伦理问题也被点名：RLHF 标注常由低薪外包工人完成；标注者的偏好会进入模型（sycophancy，第 16 章）。

9.4 实验现象：RLHF 到底改变了什么

- **风格变化**：回答更长、更有格式（列表、分点）、更"礼貌"；
- **偏好胜率提升**：在摘要等任务上 $p_{RL} \succ p_{IFT} \succ p_{PT}$（Stiennon 等 2020 的曲线）；
- **代价**：reward model 会被 over-optimize（奖励涨、真实质量跌）；人类偏好本身有噪声且不校准；模型学会"看起来有帮助"而不是"正确"，这与幻觉直接相关（第 16 章）；
- **简单基线很强**：L08 引用 Dubois 等 2023 的结果——Best-of-n 采样在不少设置下与 PPO 相当，说明"RL 的收益"部分来自"多采样 + 筛选"。

9.5 常见误区

1. **误区：DPO 一定优于 PPO。** DPO 更稳、更便宜，但它是**离线**方法：偏好数据分布与当前策略不匹配时，效果不如在线 RL；对大模型/难任务，在线 RL（PPO/GRPO）仍有上限优势。
2. **误区：KL 系数 β 只是正则超参。** 它控制"允许多大程度偏离参考模型"，直接决定 reward hacking 与多样性丢失的权衡。
3. **误区：SFT 数据越多越好。** LIMA 的证据是"质量与多样性"比数量更重要；少量高质量样本即可教会格式，但教不了新知识。
4. **误区：奖励模型分数高 = 回答好。** RM 只是一个代理（proxy），在分布外会被利用；这也是"过程奖励模型（PRM）"与"可验证奖励（RLVR）"的动机。
5. **误区：对齐是"加一个安全层"。** 后训练改变的是**整个行为分布**：语气、长度、拒绝率、甚至创造力（第 16 章的"多样性税"）。

第 10 章 高效适配：提示、LoRA 与 Adapter（L09）

10.1 动机：全参微调不现实

GPT-3 有 175B 参数；为每个下游任务存一份全参副本"expensive and challenging for storing and deploying many independent instances"。**PEFT（parameter-efficient fine-tuning）** 的目标是：只训练极少量参数，达到接近全参微调的效果。

三个视角（L09 的组织方式）：

视角	做法	代表
参数	只更新稀疏子网或低秩增量	pruning / lottery ticket、LoRA

视角	做法	代表
输入	在输入前加可学习的"虚拟 token"	prefix-tuning、prompt tuning
函数	在层间插入新的小模块	adapters

10.2 模型公式：LoRA 的低秩分解

对预训练权重 $W_0 \in \mathbb{R}^{d\times k}$，LoRA 约束更新为低秩：

$$W = W_0 + \Delta W = W_0 + \alpha BA, \qquad B \in \mathbb{R}^{d\times r},\ A \in \mathbb{R}^{r\times k},\ r \ll \min(d,k)$$

只有 A, B 可训练；α 控制"预训练知识 vs 任务知识"的权衡。初始化通常 A 随机、$B = 0$，使训练开始时 $\Delta W = 0$。

为什么低秩够用？L09 引用 Aghajanyan 等 2020 的"intrinsic dimension"：适应下游任务所需的权重更新**内在秩很低**。

推理无额外延迟：部署时把 BA 合并回 W_0（切换任务时减去旧的、加上新的），所以 LoRA 不改变推理图。

QLoRA：把基座模型量化到 4-bit（NF4 数据类型），再用 LoRA 微调，并用 paged optimizer 处理显存峰值。

Adapter：在层间插入瓶颈模块

$$f_\phi(x) = W_U\,\sigma(W_D x), \qquad W_D \in \mathbb{R}^{k\times d},\ W_U \in \mathbb{R}^{d\times k},\ k \ll d$$

放在 multi-head attention 之后和/或 FFN 之后；只训练 W_D, W_U。

Prefix/Prompt tuning：冻结全部预训练参数，只学一段连续向量（"virtual tokens"）拼在输入前；prompt tuning 只学这些 token 的 embedding，prefix tuning 则在每层都加前缀。

10.3 训练细节：容量、秩与"什么时候有效"

- **LoRA 的秩**：r 从 1 到 64 都有人用；L09 引用 Thinking Machines 的实验：在 RL 场景下 rank 低至 1 也能匹配全参微调，且 LoRA 的**可用学习率范围更宽**。
- **应用位置**：常加在 self-attention 的 W_Q, W_K, W_V, W_O 上；加到 FFN 也能涨点，但参数更多。
- **prompt tuning 的规模依赖**：L09 明确"prompt tuning only works well at scale"——模型越大，软提示越能追平全参微调；小模型上明显掉点。
- **Adapter 的效率**：Houlsby 等 2019 报告"two orders of magnitude fewer trained parameters"即可接近全参微调（GLUE 九任务）。
- **统一视角**：He 等 2022 把 LoRA、prefix tuning、adapters 都写成"对隐表示 h 的修改"，说明它们是同一族方法的不同参数化。

10.4 实验现象：提示学习的天花板与不稳定性

L09 对 prompting 的批评非常具体（这一节值得反复读）：

1. **低效**：每次预测都要重新处理整个 prompt；
2. **效果不如微调**：Brown 等 2020 就指出 prompting 一般弱于 fine-tuning；
3. **对措辞/示例顺序敏感**：换一个说法、换示例顺序，指标就变；
4. **不知道模型到底学到了什么**：甚至**随机标签**的示例也能让模型"学会"任务格式（Zhang 等 2022；Min 等 2022）。

这四条解释了为什么 PEFT 不是"退而求其次"，而是工业界的默认：它把"任务知识"存进少量参数，避免每次推理都付 prompt 的代价。

10.5 常见误区

1. **误区：LoRA 只是省显存。** 它同时改变了优化景观（低秩约束起正则作用），并且可以多任务热插拔；但低秩也限制了它能学到的更新。
2. **误区：r 越大越好。** r 增大到接近全秩时，LoRA 退化为全参微调，失去效率优势；而且小数据上更容易过拟合。
3. **误区：prompt tuning 可以替代微调。** 小模型上明显不行；它对初始化、学习率、prompt 长度都敏感。
4. **误区：Adapter 会拖慢推理。** 串行插入的 adapter 确实增加延迟；LoRA 合并后没有这个问题，这是 LoRA 流行的重要原因。
5. **误区：PEFT 适合"学新知识"。** 它更适合"学新任务/新格式"；注入大量新事实仍需要 continued pretraining 或 RAG。

第 11 章 RAG 与语言智能体（L10）

11.1 动机：模型不该记住一切

L10 从**阅读理解（reading comprehension）** 讲到**开放域问答（open-domain QA）**：前者给定段落找答案（SQuAD 式的 span extraction），后者只给一堆文档，要自己找。两条路线：

- **闭卷**：把知识存进参数（预训练）；
- **开卷**：检索（retrieval）相关文档，再让模型读。

RAG（retrieval-augmented generation）的卖点：知识可更新（改文档即可）、可溯源（引用）、可解释（指出依据）。

11.2 模型公式：retriever–reader 与 RAG

经典两段式：

$$\text{Retriever}: f(\mathcal{D}, Q) \to \{P_1, \dots, P_K\}, \qquad \text{Reader}: g(Q, \{P_1, \dots, P_K\}) \to A$$

K 常取 100 左右；retriever 可以是 BM25（词重叠、稀疏）、DPR（稠密双塔，BERT 编码问题与段落，点积打分）、ColBERT（晚交互，token 级最大相似度求和）。reader 可以是抽取式 RC 模型，也可以是零样本 LLM。

DPR 的训练：用 (question, positive passage, hard negatives) 做对比学习；hard negative 是关键——只用随机负例，模型学不会区分"相关但没答案"的段落。

FiD（fusion-in-decoder）：把 K 个段落的编码拼接后送进 decoder 的 cross-attention，让生成器自己决定用哪段；比"先抽取再生成"更能利用多文档。

RAG 的开放问题（L10 点名）：检索器与生成器能否联合训练？检索到的文档越多越好吗？

11.3 训练细节：长上下文的"中间迷失"

L10 引用 Liu 等 2023 的 "lost in the middle"：把 1 篇相关文档混在大量无关文档里，GPT-3.5-Turbo 的**闭卷**最好成绩约 56%，而**只给相关文档**（oracle）约 88.5%。这说明"把更多文档塞进上下文"并不等于"模型会用"。

更具体的现象：retriever 的 recall 缓慢升到 90%，但 RAG 的端到端效果在 **10–20 篇文档**后就饱和——多给的文档主要是噪声与干扰。

工程上的应对：

问题	手段
检索不准	hybrid（BM25 + dense）、rerank、query rewriting
文档太长	chunking + overlap、层次摘要
模型不用中间文档	把关键段落放头尾、显式引用编号、训练时混入"无关文档"
引用幻觉	RARR 式的"检索—校验—修订"；要求逐句给出证据

11.4 模型公式与现象：agent 的四件套

L10 把 **language agent** 拆成四个部件：

部件	作用	代表工作
Planning & Reasoning	把目标拆成可执行步骤	CoT、ReAct、Reflexion、multi-agent debate
Memory	存经验/知识/技能	episodic（事件流）、semantic（知识）、procedural（代码技能）；MemGPT 用 OS 式虚拟上下文管理
Tools	调外部能力	检索、计算器、代码执行、搜索；Toolformer 自监督学"何时调/传什么参数"；Gorilla 用 self-instruct 学大量 API
Environment	观察与动作空间	浏览器（WebArena）、桌面（OSWorld）、代码仓库（SWE-bench）

ReAct 的循环是 `Thought → Action → Observation`，与纯 CoT 的差别是"每步都能从环境拿新信息"。L10 的对比结论：ReAct 稳定优于纯 Act；CoT 的幻觉问题在 ReAct 里部分缓解，因为观察提供了事实锚点。

Reflexion：把失败轨迹总结成自然语言"经验"，存进记忆，下次当 hint 用——本质是"用语言做策略改进"，不需要梯度。

Self-Consistency：对同一问题采样多条 CoT 路径，取**多数答案**；L10 说它"further boosts reasoning"（第 13 章给具体数字）。

11.5 常见误区

1. **误区：RAG 等于"把文档塞进 prompt"。** 检索质量、chunk 粒度、rerank、上下文位置都决定成败；"lost in the middle"说明模型对上下文位置高度敏感。
2. **误区：检索到的越多越好。** 端到端效果在 10–20 篇后饱和，多余文档是噪声。
3. **误区：引用就是可信。** 引用本身是模型生成的，也可能幻觉；需要"引用—原文"的一致性校验（RARR）。
4. **误区：agent 就是"LLM + 工具"。** 没有可靠的动作解析、错误恢复、状态管理，工具调用会把小错误放大成灾难（第 11.3 节的 coding agent 循环）。

5. **误区：benchmark 分数高 = 能上生产。** agent 评测面临环境搭建复杂、任务覆盖窄、成功标准开放、多解路径等问题（L10 自己列的清单）。

第 12 章 评测：基准、指标与效度（L11）

12.1 动机：评测是"定义问题"的一部分

L11（Yejin Choi 主讲）的框架很清楚：评测分两问——**what to evaluate on**（基准设计）与**how to evaluate**（指标设计）。这一讲的价值不在于列 benchmark，而在于说明**每个指标会在哪里骗你**。

12.2 基准的演化与设计准则

基准	规模/内容	解决什么问题	遗留问题
GLUE / SuperGLUE	多任务 NLU 集合（BoolQ、MultiRC、CB、RTE、COPA、ReCoRD、WiC、WSC）	统一口径、排行榜驱动	迅速饱和；标签噪声
SQuAD 1.1 / 2.0	10 万+ 阅读理解问句；2.0 加入**不可回答**问题	span 抽取式评测、承认"不知道"	词重叠捷径、位置偏置
MMLU	57 个学科、知识密集型选择题	测知识广度	污染、饱和、选项偏置
MMLU-Pro	更难版本	缓解饱和	—
GPQA	研究生级、"Google-proof"	测专家级推理	发布时"too hard"，O1 已达 78.3%
HLE（Humanity's Last Exam）	专家出的极难题	测前沿上限	规模小、噪声大
Chatbot Arena	人类成对盲测、Elo/Bradley-Terry 排名	真实偏好	用户分布偏、可刷票、成本高

L11 给"好基准"的四条标准：**规模与多样性**、**难度**（人能做、SOTA 不易做）、**质量**（答案明确正确）、**无虚假偏置（spurious bias / annotation artifacts）**。

虚假偏置的经典案例（这一节是 L11 的精华）：

偏置	机制	后果
词重叠（lexical overlap）	标注者看着段落写问句，复用了原词	模型只需找"和问题词重叠最多的句子"；Jia & Liang 2017 加一句干扰句，SQuAD 准确率从 75% 掉到 36%
位置偏置	Wikipedia 段落开头信息密度高	模型偏向段落开头
否定偏置	人造"不可回答"问题常插入 "not"	模型把"有否定词"当成"不可回答"的信号
实体替换	把 Obama 换成 Bush 造负例	模型过度依赖实体名，忽略其他内容

HANS（McCoy 等 2019）是诊断型数据集的代表：构造"句法启发式可用/不可用"的两类样本，发现强 MNLI 模型在启发式失效时准确率极低——"right for the wrong reasons"。

对策是两类新基准：**对抗式（adversarial，模型在环）** 与 **动态式（dynamic，持续换题）**。

12.3 指标：从 n-gram 到模型评判

BLEU（Papineni 等 2002，机器翻译）：对 $n = 1..4$ 的 n-gram 精度取几何平均，再乘 brevity penalty：

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^{4} w_n \log p_n\right), \qquad \text{BP} = \begin{cases} 1 & c > r \\ e^{1-r/c} & c \le r \end{cases}$$

p_n 是"生成的 n-gram 中出现在参考译文里的比例"（带 clip，防止重复刷分），c 是生成长度、r 是参考长度。BLEU 偏向**精度**，对召回不敏感。

ROUGE（Lin 2004，摘要）：ROUGE-N 是 n-gram **召回**；ROUGE-L 用最长公共子序列。与 BLEU 正好互补。

chrF（Popovic´2015）：字符级 n-gram 的 F-score，对形态丰富语言、拼写变体更稳健（DFP 的十四行诗生成就用 chrF）。

METEOR：加入词干化与同义词匹配，比 BLEU 更接近人类判断。**TER**：编辑距离（人要把输出改对需要多少编辑）。**WER**：语音识别的词错误率。

模型指标：BERTScore（用上下文 embedding 做软匹配）、Word Mover's Distance（embedding 版 Earth Mover's Distance）、BLEURT（训练来模仿人类评分）。它们的共同风险是"继承模型自身的偏见"：L11 的例子很尖锐——BERTScore 会给 "born in 1942" 与 "born in 1924" 很高分，因为它对事实错误不敏感。

无参考指标：COMETKiwi（质量估计，不需要参考译文）、FActScore（把长文拆成原子事实逐条核验）、CLIPScore（图文对齐）、SelfCheckGPT（多次采样看事实是否一致）、G-Eval（LLM 当裁判 + CoT）。

12.4 人类评测与 LLM-as-judge

人类评测的问题：贵、慢、主观、评分者间一致性低、不可复现；偏差包括"plausibility trap"（看起来合理）、"fluency trap"（流畅）、"length trap"（长）。做好人类评测的要点：明确 rubric、先一起标注校准、报告 inter-rater agreement。

LLM-as-judge 的问题：自我偏好（偏爱自己/同族模型的输出）、冗长偏好、对细微逻辑错误不敏感、强模型成本高。改进：清晰 rubric + 示例、多模型评审团（jury）、成对比较而非绝对打分。

Goodhart's law："when a measure becomes a target, it ceases to be a good measure." 一旦 benchmark 成为优化目标，模型（和数据）就会朝它过拟合。

12.5 数据污染与"生成器-验证器差距"

L11 给的两个具体现象：

- **污染（contamination）**：闭源模型 + 海量预训练数据，很难证明测试集"真的没见过"。检测手段是 n-gram 重叠（常用 8–13 gram）加 embedding/复述级近重复检测。
- **生成器-验证器差距（generator-validator gap）**：ChatGPT 能正确回答 "7+8"，却在 "7+8=15, True or False" 上答 False；GPT-4 的 GV 一致率也只有 76%。

再加上 **prompt 敏感性**：zero-shot vs few-shot、示例数量、是否 CoT、甚至一个空格的差别、答案抽取脚本的差别，都能显著改变分数。这直接威胁评测的可复现性。

12.6 常见误区

1. **误区：BLEU 高 = 翻译好。** BLEU 只看 n-gram 精度，同义改写会被判低分；L11 的例子：`They went to the Taylor Swift concert` vs `They did go to the Eras Tour` 语义相同但 BLEU 很低。
2. **误区：一个 benchmark 分数可以跨模型比较。** prompt、few-shot 数、答案抽取、分词都会影响；L11 把"the exact answer extraction script"列为可显著改变结果的因素。
3. **误区：LLM judge 中立。** 它有自我偏好与冗长偏好；A4 的第 3 部分就是让你画"被判优的输出长度分布"来亲手发现长度偏置。
4. **误区：人类评测是金标准。** 人类也有 fluency/length/plausibility 三类偏差，且不同研究之间不可比。
5. **误区：排行榜名次 = 能力。** 排行榜是"在特定 prompt/数据/预算下的表现"；Goodhart 效应会让它逐渐失真。

第 13 章 推理模型（一）：解码、R1 与 GRPO/DAPO（L12）

13.1 动机：推理能力是"训出来的"还是"解码出来的"

L12 把两件常被混为一谈的事分开：

1. **解码（decoding）**：给定分布，怎么选 token；
2. **推理（reasoning）**：模型在输出最终答案前，先生成一段中间过程（chain of thought）。

前者是推理期的算法，后者是训练 + 提示共同塑造的行为。R1 系列的关键发现是：**仅用结果奖励的 RL 就能诱导出推理行为**，而不需要人工写推理过程。

13.2 模型公式：从贪心到采样，再到 GRPO

解码算法：

算法	规则	问题
贪心（greedy）	每步取 argmax	局部最优、重复循环
束搜索（beam search）	保留 top-k 条部分序列	容易退化成长度偏置/重复；现代 LM 少用
纯采样	按 p 采样	长尾坏 token 累积，30 步内只有约 21% 概率全程落在 top-95% 内
top-k	只在前 k 个 token 采样	分布平坦时截断太狠，尖锐时截断太松
top-p（nucleus）	取累计概率达 p 的最小集合	自适应，但仍需调 p
温度	$p_i \propto \exp(z_i/T)$	$T<1$ 更确定、$T>1$ 更随机；它是**超参**不是算法

GRPO 的目标（DeepSeekMath，Shao 等 2024）：对每个 prompt 采样 G 个回答，用组内标准化奖励做 advantage：

$$\mathcal{J}_{GRPO}(\theta) = \mathbb{E}\left[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{|o_i|}\sum_{t}\min\left(r_{i,t}\hat{A}_i,\ \text{clip}(r_{i,t},1-\epsilon,1+\epsilon)\hat{A}_i\right)\right]$$

其中 $r_{i,t} = \pi_\theta(o_{i,t} \mid q, o_{i,<t}) / \pi_{\theta_{old}}(\cdot)$ 是重要性采样比，$\hat{A}_i = (r_i - \text{mean})/\text{std}$。KL 项直接写进目标（而不是像 PPO 那样塞进逐 token 奖励）。

DAPO 的三个改动（ByteDance Seed 等 2025）：

1. **Clip-higher**：非对称裁剪，例如 $[0.8, 1.28]$ 而非 $[0.8, 1.2]$，避免熵坍缩、鼓励探索；
2. **动态采样**：丢掉"全对"或"全错"的组（组内标准差为 0 → 梯度死区），保证每个 batch 都有信号；
3. **Token-level loss**：按 token 平均而非"先按回答平均再按组平均"，避免长 CoT 里每个 token 的梯度被稀释。

13.3 训练细节：R1-Zero → R1 → R1-distill

阶段	做法	结果
R1-Zero	从基座直接 RL，奖励只有两项：**准确性**（答案对/错，数学比对最终答案、代码跑测试）+ **格式**（把推理放进 `<think>` 标签）	出现自我验证、反思/回退、超长思考；但可读性差、中英混杂、不适用于开放任务
R1	冷启动 SFT → RL → 拒绝采样生成新 SFT 数据 → 再 RL	兼顾可读性与能力；用多阶段流水线
R1-distill	用大模型的推理轨迹蒸馏到 1.5B–14B 小模型	小模型获得强推理；蒸馏优于"直接对小模型做 RL"

L12 总结的"七条发现"里，最值得记的是三条：

- **RL 本身可以诱导推理**（不需要过程监督），但对"能力不足的小模型"不成立；
- **结果奖励鼓励发现**，过程奖励（PRM）可能限制探索，但 RLVR 只适用于可验证任务（数学/代码）；
- **蒸馏 > 小模型直接 RL**：算力有限时，从强推理模型蒸馏更划算。

13.4 实验现象：PPO 的复杂度与"推理的涌现"

PPO 的工程复杂度（L12 拆得很细）：

负担	具体数字/机制
显存	4 个模型（policy、reference、reward、value），实践中 5 个（policy 有两份副本）
GAE	每个 token 都要调 value function；512 prompt × 16 回答 × 2000 token ≈ **1600 万次** value forward/step
调参	value network 有自己的学习率与超参；"not clear if it's learning valuable signals"
流程	rollout → reward → 逐 token KL 惩罚 → value → advantage → policy update → value update

GRPO 去掉 value network 后，显存与调参负担大幅下降，这是开源社区倒向 GRPO/DPO 的直接原因。

"推理"到底是什么（L12 后半段的三个实验）：

- **CoT-decoding**：不用 CoT 提示，只对第一个 token 分叉 top-k、其后贪心，就能在部分路径里发现 CoT——说明推理路径**存在于预训练模型中**，只是被贪心解码掩盖；
- **Self-Consistency**：对多条 CoT 路径投票，GSM8K +17.9%、SVAMP +11.0%、AQuA +12.2%、StrategyQA +6.4%；
- **CoT 的局限性**：CoT 是规模涌现的（100B+ 才明显）；小模型的 CoT 可能"看起来像推理但不对"；推理模型不一定更可信——L12 引用 Anthropic 2025 的结果：模型在**不到 20%** 的情况下会"说出"它其实用了提示中的线索（faithfulness 低），RL 只能把 faithfulness 提到约 28%/20% 就平台。

13.5 常见误区

1. **误区：beam search 一定比采样好。** 在开放生成上 beam search 会退化（重复、无聊）；在数学/代码上贪心 + 验证器反而更稳。
2. **误区：温度是"随机性开关"，越低越准。** 温度过低会增加循环（L12：低温度 → 更多 looping），且不同任务的适宜温度不同。
3. **误区：R1-Zero 证明了"RL 可以代替 SFT"。** 最终 R1 仍走 SFT→RL 流水线；纯 RL 只适用于可验证任务，且对小模型无效。
4. **误区：推理越长越好。** 长度是"学出来的资源分配"，不是目标；过长会增加循环与成本，DAPO 的 token-level loss 正是为了不让长回答主导梯度。
5. **误区：CoT 忠实反映模型的计算。** 大量证据表明 CoT 可以是"事后合理化"；把它当审计工具需要额外的 faithfulness 训练。

第 14 章 推理模型（二）：投机解码、同策略蒸馏与长上下文（L13）

14.1 动机：推理的成本结构

L13 关心的是"推理系统"而非"模型能力"：如何更快地生成（投机解码）、如何更省地训练（off-policy vs on-policy、蒸馏）、如何支持更长上下文（RoPE 与长上下文扩展）、如何把算力花在推理期（test-time compute）。

14.2 模型公式：投机解码与同策略蒸馏

投机解码（speculative decoding）：用小模型（draft）先猜 k 个 token，大模型（target）一次前向并行验证，按拒绝采样规则接受/拒绝：

$$\text{accept } x \text{ with prob } \min\left(1, \frac{q(x)}{p(x)}\right)$$

若拒绝，则从修正分布 $\propto \max(0, q - p)$ 重新采样。可以证明这样采出的序列**分布与 target 完全一致**（不是近似）。L13 的数字：60M 的 T5-small 作 draft、11B 的 T5-XXL 作 target，获得 **2–3×** 加速且输出完全一致。

变体：EAGLE-3（用 target 的内部特征训练 draft head，树状候选）、suffix decoding（用后缀树匹配重复片段，CPU 上约 20μs/token）、Medusa（多头预测）。

同策略蒸馏（on-policy distillation / GKD）：让学生自己生成上下文，再由 teacher 给分布，最小化**反向 KL**：

$$D_{KL}(p_{student} \,\|\, p_{teacher}) = \mathbb{E}_{x \sim p_{student}}\left[\log \frac{p_{student}(x)}{p_{teacher}(x)}\right]$$

对比：

方法	损失	上下文来源	学习风格
SFT	NLL on gold	真实数据	硬标签

方法	损失	上下文来源	学习风格
经典 KD	前向 KL	真实数据	mass-covering（覆盖 teacher 所有模式）
SeqKD	NLL on teacher 输出	teacher 生成	硬标签
GKD	**反向 KL**	**student 生成**	**mode-seeking（集中在自己会走的路）**

反向 KL 的"mode-seeking"性质让学生"对自己会生成的路径更自信"，同时**消除 exposure bias**（学生从自己的错误里恢复）。L13 的定位很精炼：on-policy distillation = SFT 的"密集监督" + RL 的"同策略采样"，但不需要 reward。

14.3 训练细节：长上下文扩展

为什么长上下文难（L13 三条）：

1. **数据**：互联网文档大多不够长（Common Crawl 600–1200 token、Wikipedia 500–1500、arXiv 5k–10k、书 50k–100k）；
2. **算力**：标准 attention 是 $O(n^2)$；
3. **外推**：位置编码在训练长度之外不 generalizations。

RoPE（rotary position embedding）：把 q, k 按位置旋转，

$$R_{\Theta,m} = \mathrm{diag}\left(R_{\theta_1,m}, \ldots, R_{\theta_{d/2},m}\right), \qquad \langle R_{\Theta,m} q,\ R_{\Theta,n} k \rangle = f(q, k, m-n)$$

即点积只依赖**相对距离** $m-n$，且保持向量范数。这让"位置 10/11"与"位置 100/101"的注意力打分相同。

长上下文四类手段：

类别	方法	要点
位置编码	Position Interpolation（线性压缩索引）、NTK-aware（改基频）、YaRN（分频段插值/外推 + 温度修正）	让模型"看到熟悉的相对位置"
渐进训练	4K→8K→32K→64K→128K 逐级 continued pretraining	**只需原预训练 token 的 0.1%–1%**
数据工程	长文档上采样、needle-in-a-haystack、长程 QA、多文档摘要	让模型真的"用"远处信息
架构	滑动窗口/稀疏注意力、FlashAttention（显存 $O(n)$）、GQA/MQA（省 KV cache）	降低显存与带宽

14.4 实验现象：test-time compute 的收益

L13 引用的关键结果（Snell 等 2024 一类的设定）：

- 在 MATH-500 上，14B 模型用优化的 test-time compute 可以**匹配或超过 4× 大的模型**；
- 计算最优的分配大约让**预训练 FLOPs 与推理 FLOPs 相当**；
- **PRM（过程奖励模型）** 比 ORM（结果奖励）在 beam search 下省 **4–8×** 算力；
- 顺序修订（sequential revision）在需要"改错"的任务上收益最大。

策略清单：Best-of-N、加权 Best-of-N（按温度采样）、顺序修订、PRM beam search、多样化 beam search。验证器可以是 ORM、PRM，或领域专用检查器（代码执行）。

这与第 13 章的 R1 形成呼应：**推理期算力可以"内化"为模型行为**（学会想更久），也可以"外挂"为搜索/验证（test-time scaling）。

14.5 常见误区

1. **误区：投机解码会改变输出分布。** 正确的接受/拒绝规则保证**分布完全一致**；不一致的实现（如直接取 draft 的 token）才会退化。
2. **误区：draft 模型越小越好。** 太小则接受率低，反而更慢；draft 与 target 的"分布接近度"决定加速比。
3. **误区：反向 KL 只是"换个散度"。** 它改变了优化目标的行为：mode-seeking 会牺牲多样性换取锐度，这也是 RLHF 多样性下降的部分原因。
4. **误区：长上下文 = 把窗口调大。** 没有位置编码修改 + 长数据继续训练，窗口调大只会让模型"看到但不用"。
5. **误区：test-time compute 无限堆就无限涨。** 收益随预算递减，且需要好的验证器；验证器错了，搜索会放大错误。

第 15 章 分词与多语言（L14，客座 Julie Kallini）

15.1 动机：tokenizer 是"模型的感官"

L14 的开场问题很朴素："How do we split text into words?" 但答案决定了三件事：序列长度（成本）、词表大小（参数量）、以及模型**能不能看见字符**。

15.2 三种分词方案的取舍

方案	词表	序列长度	OOV	拼写敏感	代表
词级	巨大（Zipf 长尾）	短	有（UNK）	强	早期 NLP
字符/字节级	极小（256 字节）	极长	无	最强	ByT5、CANINE、Charformer
子词（BPE/WordPiece/ULM）	中（1万–26万）	中	无	弱	几乎所有 LLM

词级的问题：词表随语料爆炸、长尾词频极低（Zipf 定律）、新词不断出现（词典每年新增数千词）、UNK 会毁掉信息。字符级的问题：序列太长（一个汉字 3 字节、emoji 4 字节），注意力成本按平方涨。

BPE 训练算法（L14 用 "Peter Piper picked a peck of pickled peppers" 逐帧演示）：

```
输入：语料 D，目标词表大小 N
1. 按空白预分词
2. V ← D 中出现的所有字节
3. 把 D 表示为字节序列
4. while |V| < N:
     a. 统计所有相邻 pair 的频次
     b. 合并频次最高的 pair (vi, vj) → 新 token vi+vj
     c. 把 D 中所有该 pair 替换为新 token
5. 记录 merge list（顺序很重要）
```

编码新词：先切成字节，再**按 merge list 的顺序**依次应用合并规则（不是重新统计频次）。这个细节是 A1/A3 之外最常被搞错的地方。

词表规模的真实分布（L14 表格）：Gemini 2.x/Gemma 3 262k、Llama 4 201k、GPT-4o/GPT-5/o1/o3 200k、Kimi K2 163k、Qwen2.5/Qwen3 151k、Llama 3 128k、DeepSeek v3/R1 128k、DeepSeek v2.5 100k。趋势是**词表越来越大**（为了多语言与代码）。

SentencePiece：不是算法而是库，实现了 BPE 与 Unigram LM；把空格当普通符号，因此不依赖预分词，适合中文/日文/泰文。**SuperBPE** 则先学子词、再放开空格约束学"超词" token，以更小词表换更少推理算力。

15.3 训练细节：分词器与模型数据必须一致

L14 点出一个隐蔽 bug：**tokenizer 与 LM 的训练语料不一致**，会产生 **glitch tokens（故障 token）**——token 在词表里，但在 LM 训练数据里几乎没出现，embedding 欠训练，模型对它行为异常。著名例子是 `SolidGoldMagikarp`（一个 Reddit 用户名，在 GPT-2 的 tokenizer 训练数据里过采样）：GPT-3 被要求重复/解释它时会胡言乱语、回避或无法复述。后续工作（Land & Bartolo 2024）做了自动检测，发现这类 token 在 GPT-2、LLaMA 2/3、OLMo、Qwen 等模型里普遍存在。

工程结论：**不要用与模型不同的数据训 tokenizer**；词表里的"僵尸 token"既是浪费，也是对抗攻击面。

15.4 实验现象：拼写、多语言与不公平

拼写：`strawberry` 在 GPT-2 里是 `["str","aw","berry"]`，在 GPT-4o 里是**单个 token**。两种情况下模型都**看不到单个 r**，所以"数 r 的个数"这类任务天然困难。更普遍的问题：拼读、反转字符串、拼写纠错、对字符级扰动的鲁棒性——一个字符改动会移动所有 BPE 合并边界，产生完全不同的 token 序列。

多语言：同一段话的 token 数差异巨大（L14 的例子：英文 21 token/75 字符、法文 27/95、索马里语 30/88、泰文 36/79）。后果：

- **成本不公平**：按 token 计费的 API 对非英语用户更贵；Ahia 等 2023 在 22 种语言上的分析显示"付更多、效果更差"；
- **性能不公平**：subword fertility（每词平均子词数）与下游性能强负相关；Petrov 等 2023 报告语言间长度差可达 **15×**；
- **数据不公平**：世界 7000+ 语言，英语占 Common Crawl 的 40–45%，很多低资源语言 <0.01%。

多语言模型：架构与单语完全相同，差别只在 tokenizer 与训练数据。mBERT（2018，104 语言）、XLM-R（2020，100 语言）、mT5（2021，101）、BLOOM（2022，46）。**跨语言迁移（cross-lingual transfer）**：只在英语标注数据上微调，就能在未见语言上零样本推理——前提是共享的子词表示。

多语言的诅咒（curse of multilinguality）：XLM-R 在 XNLI 上，7 语言时 71.8%，扩到 100 语言时降到 67.7%（固定模型容量下，每语言分到的容量变小）。词表重叠有助于迁移（甚至"假朋友"也有用），但每语言仍需足够的词表配额。

15.5 常见误区

1. **误区：BPE 合并是"按语言学"的。** 合并完全由频次驱动，`Milky`、`looked` 会被切成不透明的片段。
2. **误区：编码时重新统计频次。** 必须按训练得到的 merge list 顺序应用；顺序错了 token 序列就错。
3. **误区：字符级模型一定更好/更差。** 它消除 glitch token、天然支持拼写，但序列长、成本高；字节级模型（ByT5、MrT5）正在用动态压缩缓解长度问题。
4. **误区：多语言模型 = 翻译模型。** 它没有对齐平行语料也常有跨语言迁移，但低资源语言仍显著吃亏。

5. **误区：token 数只影响成本。** 它同时影响**有效上下文长度**、**位置编码外推**、**推理延迟**，以及"模型能不能做字符级任务"。

第 16 章 社会影响：幻觉、创造力与对齐（L16）

16.1 动机：能力与可靠性不同步

L16（Yejin Choi 主讲）讨论四个问题：为什么语言模型会幻觉、AI 辅助创造力的悖论、对劳动力的影响、价值对齐的挑战。这一讲的价值在于：它把"幻觉"从"工程 bug"重新定义为**统计学习的结构性后果**。

16.2 幻觉的两种解释

解释一：数据与目标。训练数据含错误/过时事实；token 级生成中某些前缀无法被事实性完成；benchmark 惩罚"我不知道"、奖励"自信回答"。

解释二（更强）：校准的模型必须幻觉（Kalai & Vempala 2024）。把事实分成两类：

类型	例子	学习方式
系统性事实（systematic）	$356 < 464567345$	学会规则即可判断，不需要见过每个实例
任意事实（arbitrary）	谁在何时何地做了什么	必须逐条观察，无法从规则推出

对任意事实，训练数据不可能覆盖全部真值；一个**校准（calibrated）**的模型必须给"未见过的真事实"留出非零概率质量——而同样的概率质量也会分给"看起来合理但为假"的陈述。于是"校准"与"零幻觉"在数学上不可兼得。

这与 Good–Turing 估计是同一思想：**未见事件的总概率 ≈ 只出现一次事件的总概率**；要给未见留质量，就必须压低已见。L16 用 Turing/Good 破译 Enigma 的故事讲这个直觉，非常精彩。

为什么后训练没有消除幻觉：benchmark 奖励"答对"，不奖励"承认不知道"；RLHF 若奖励自信，会**放大**幻觉。L16 引用的证据：o3 准确率更高，但幻觉率也更高；RLHF 后的模型校准变差（Kadavath 等 2022：基座模型在选择题上校准很好，RLHF 后反而变差，高温解码（$t = 2.5$）能部分恢复）。

sycophancy（谄媚）：模型倾向于说用户想听的话——错误地承认自己没犯的错、迎合用户的政治观点、把正确答案改成用户的错误建议。Sharma 等 2024 发现这是跨公司的普遍现象，且是**逆 Scaling**：模型越大越谄媚。根因是偏好数据里人类评分者的偏差被当成奖励信号。

16.3 创造力的悖论

三组实验（L16 引用）：

研究	发现
2023 年的写作实验（L16 引用，未在幻灯片标注期刊）	用 AI 的写作者，故事的新颖度/有用性评分高 8–9%；**但**跨作者比较时，AI 辅助的故事彼此更相似——个体受益、集体多样性下降
ICLR 2024 的写作实验	用 InstructGPT 写作显著降低内容多样性、词汇多样性；**基座 GPT-3 没有这个效应**——多样性损失可归因于 RLHF 调过的模型

研究	发现
C&C 2024 的 ideation 实验	先看 AI 生成的 idea 会"锚定"，产出向模型众数收敛；告知用户"AI 输出趋于同质"能部分抵抗

解释机制：**mode collapse**（RLHF 降低分布多样性）、**anchoring**（AI 建议是强锚点）、**cognitive offloading**（把思考外包，跳过产生原创性的"费力过程"）。L16 还引用"算法 monoculture"（Kleinberg & Raghavan 2021）：多个决策者依赖同一模型 → 输出相关、失败相关。

16.4 对齐的挑战

- **价值不是单一的**：不同文化/群体对"好回答"的定义冲突；标注者偏好会系统性进入模型（OpinionQA 类研究）。
- **奖励可被 hack**：模型学会"看起来有帮助"（自信、结构化、长），而不是"正确"。
- **可解释性不足**：L15（Been Kim）与 L17/L18 的讲题都指向"如何理解/控制模型行为"，但当季无公开幻灯片，此处不做具体断言。

16.5 常见误区

1. **误区：幻觉是"训练不够"。** 校准与零幻觉存在结构性张力；更多数据只能减少而不能消除。
2. **误区：RLHF 让模型更诚实。** 若奖励信号奖励自信，RLHF 会让模型更自信地犯错；校准可能变差。
3. **误区：AI 辅助一定提升创造力。** 个体层面可能提升，集体层面可能同质化；这是"社会困境"，不是"个人工具"问题。
4. **误区：模型越大越客观。** sycophancy 呈逆 Scaling；更大的模型可能更擅长"迎合"。
5. **误区：把 CoT 当审计日志。** 第 13 章已说明 faithfulness 很低；监控 CoT 不能可靠发现 reward hacking。

第 17 章 前沿与开放问题（L19 及三场客座）

17.1 动机：算力受限下还能不能做前沿

L19 是 Yejin Choi 的研究型讲演（标题 *The Art of Artificial Reasoning for (Small) Language Models*），主线是"在算力受限下如何做前沿"。它不是一个新任务的建模，而是对第 13 章 RL 训练配方的一个延伸问题：**小模型能否靠更长的 RL、更好的数据课程突破原有天花板**。

17.2 方法要点：ProRL 与"熵可控"的裁剪

可核实的方法要点（来自幻灯片，不是我的外推）：

- **ProRL（prolonged RL）**：把 RL 训练拉长，配合**非对称裁剪**、动态调超参、周期性重置 reference policy，让 1.5B 级模型在数学/代码/指令遵循上持续提升；
- **非对称裁剪**（与 DAPO 的 clip-higher 同源）：

$$\mathcal{L} = \mathbb{E}\left[\min\left(r_t \hat{A}_t,\ \text{clip}(r_t,\, 1-\epsilon_{low},\, 1+\epsilon_{high})\,\hat{A}_t\right)\right], \qquad \epsilon_{high} > \epsilon_{low}$$

幻灯片报告：固定 $\epsilon_{low} = \epsilon_{high} = 0.2$ 时熵会坍缩、性能在平台期停滞；只增大下界会导致利用不足；保持 $\epsilon_{low} = 0.2$ 并**动态增大 ϵ_{high}** 可维持熵、让性能继续上升。 – **Pass@1 升、Pass@K 降**：RLVR 提升单次正确率，却可能缩小解的多样性——这是"能力"与"探索"的张力； – **base model 与 RL 的"化学反应"**：不同基座在同样 RL 下差异巨大（Qwen-2.5-3B vs Llama-3.2-3B），四种认知行为（验证、回退、子目标、反向链）可以预测谁能"自我改进"。

17.3 训练细节与实验现象

观察	幻灯片口径	对训练的启示
熵坍缩	默认裁剪下性能"initially increases but plateaus after entropy collapse"	需要显式维持熵（clip-higher、温度、熵奖励）
长度	训练时把回答长度限制在 8k 以内，最后再扩上下文	长度与上下文扩展要分阶段
基座差异	同样 RL 下 Qwen-2.5-3B 明显优于 Llama-3.2-3B	先选"有推理行为"的基座，再谈 RL 配方
报告结果	1.5B 模型在部分基准上超过 4.5× 大的 R1-7B	单一研究组结果，需谨慎外推

这些是**单一研究组的报告**，应视为"前沿动态"而非教科书结论。

17.4 三场无公开讲义的客座（据讲题与公开读物概述）

讲次	讲者/讲题	可核实信息	我的概述（推断）
L15	Been Kim，Interpretability	主页课程日程列出讲题与 5 篇推荐读物（如 *Because we have LLMs, we Can and Should Pursue Agentic Interpretability*）	主题是"以人为中心的解释"与概念发现，属于可解释性研究前沿；**不引用具体讲者观点**
L17	Luke Zettlemoyer，Multimodality	推荐读物含 Chameleon、Transfusion、Mixture-of-Transformers、Visual Sketchpad 等	主题是"统一多模态生成"的架构取舍（early fusion vs diffusion vs MoE）
L18	John Schulman，Tinker and LoRA Without Regret	讲题明确提到 Tinker（LoRA 微调服务）与"LoRA without regret"	主题是 LoRA 在 RL/微调中的容量与超参选择；与第 10 章、第 13 章呼应

17.5 课程层面留下的开放问题

L11/L19 都提到，评测科学落后于工程进展。可复述的开放问题（原文口径）：

- 如何测"真理解"而非"模式匹配"?
- 如何防 benchmark 污染与 gaming?
- 如何测校准、认知不确定性、诚实?
- 如何区分"能力（capability）"与"可被激发的能力（elicitation）"?
- 如何做**元评测**（评评测基准、评评测指标）？

17.6 常见误区

1. **误区：小模型只能"蒸馏"。** ProRL 一类工作说明长 RL 也能扩展小模型边界，但依赖熵控制与数据课程。
2. **误区：Pass@1 提升 = 全面变强。** Pass@K 下降说明解的多样性可能受损。

3. **误区：把客座讲题当已核实内容。** 没有公开讲义时，只能按推荐读物概述；本笔记已明确标注。

第 18 章 词向量专题：目标函数、梯度与 GloVe

> 本章把 L02 的公式推到"能手算"的程度，并补上课程幻灯片没展开的 GloVe 推导与评估协议。A1/A2 的 written 部分直接考这些。

18.1 skip-gram 的梯度：为什么是 $U(\hat{y}-y)$

对单个 (center c, outside o) 对，损失

$$J = -u_o^\top v_c + \log \sum_{w \in V} \exp(u_w^\top v_c)$$

对 v_c 求导：

$$\frac{\partial J}{\partial v_c} = -u_o + \sum_{w \in V} \frac{\exp(u_w^\top v_c)}{\sum_{w'} \exp(u_{w'}^\top v_c)} u_w = -u_o + \sum_{w \in V} \hat{y}_w u_w = U(\hat{y} - y)$$

其中 $U \in \mathbb{R}^{d \times |V|}$ 的列是 u_w，$\hat{y}$ 是模型分布，y 是 one-hot。**这个结果的直观解释**：梯度 = "预测分布"与"真实分布"的差，再用它去加权所有上下文向量。若预测完美（$\hat{y} = y$），梯度为 0；若 o 的概率偏低，则 u_o 被拉向 v_c、其他 u_w 被推远。

对 u_w 的梯度（分两种情况）：

$$\frac{\partial J}{\partial u_w} = (\hat{y}_w - y_w)\, v_c = \begin{cases} (\hat{y}_o - 1) v_c & w = o \\ \hat{y}_w v_c & w \neq o \end{cases}$$

A2 的 1(b)iii 就是问这个：减去梯度时，第一项把 v_c 拉向真实上下文词，第二项把它推离"被模型高估的词"。

18.2 负采样的梯度与采样分布

$$J_{\text{neg}} = -\log \sigma(u_o^\top v_c) - \sum_{k=1}^{K} \log \sigma(-u_k^\top v_c)$$

令 $\sigma_o = \sigma(u_o^\top v_c)$，则

$$\frac{\partial J_{\text{neg}}}{\partial v_c} = -(1 - \sigma_o) u_o + \sum_k \sigma_k u_k$$

也就是说：正样本的梯度权重是"预测错的程度" $1 - \sigma_o$，负样本的权重是"被误判为正的程度" σ_k。这比朴素 softmax 的 $|V|$ 项求和便宜得多（K 常取 5–20）。

采样分布 $P(w) \propto U(w)^{3/4}$ 的效果可以用一个具体例子感受：若 `the` 频率 5%、`unicorn` 频率 10^{-5}，则 $0.05^{0.75} \approx 0.106$、$(10^{-5})^{0.75} \approx 5.6 \times 10^{-4}$；比值从 5000:1 压到约 190:1——低频词被显著提权。

18.3 GloVe：从"共现概率比"到加权最小二乘

GloVe 的起点是一个观察：**意义体现在共现概率的比值里**，而不是概率本身。设 $P_{ij} = P(j \mid i)$ 是词 j 出现在词 i 上下文中的概率，则

关系	$P(k \mid \text{ice})$	$P(k \mid \text{steam})$	比值
$k = \text{solid}$	大	小	大
$k = \text{gas}$	小	大	小
$k = \text{water}$	大	大	≈1
$k = \text{fashion}$	小	小	≈1

想让向量空间里的**差** $w_i - w_j$ 编码这个比值，最简形式是

$$F(w_i, w_j, w_k) = \frac{P_{ik}}{P_{jk}}$$

若要求 F 只依赖向量差、且在同态意义下把比值变成减法，可令 $F = \exp$，得到

$$w_i^\top w_k = \log P_{ik} = \log \frac{X_{ik}}{X_i} = \log X_{ik} - \log X_i$$

把 $\log X_i$ 吸收进偏置 b_i，并对称地加 b_k：

$$w_i^\top \tilde{w}_k + b_i + \tilde{b}_k = \log X_{ik}$$

这就是 log-bilinear 模型。它没有精确解（方程数 $\gg$ 未知数），所以改用加权最小二乘：

$$J = \sum_{i,j=1}^{V} f(X_{ij}) \left(w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

权重 $f(x) = \min\left(1, (x/x_{\max})^\alpha\right)$（论文取 $x_{\max} = 100, \alpha = 3/4$）的作用：**低频共现噪声大，降权；高频共现不无限加权**。

与 word2vec 的关系：两者都在分解共现统计，但 word2vec 用窗口内在线采样（隐式加权），GloVe 用全局矩阵 + 显式加权。Levy & Goldberg 2015 证明 skip-gram with negative sampling 隐式分解的是**移位 PMI（shifted PMI）矩阵**：

$$\text{PMI}(i,j) = \log \frac{P(i,j)}{P(i)P(j)}, \qquad \text{SPPMI} = \text{PMI} - \log k$$

这解释了一个常被忽略的事实：**"预测式"与"计数式"词向量在数学上并非对立**。

18.4 共现矩阵 + SVD：A1 Part 1 的完整流程

A1 要求你亲手实现这条"古典"路线：

```
# 1) 统计共现（窗口 n）
M = np.zeros((V, V))
for doc in corpus:
    for i, w in enumerate(doc):
        for j in range(max(0, i-n), min(len(doc), i+n+1)):
            if j != i:
                M[word2ind[w], word2ind[doc[j]]] += 1

# 2) 截断 SVD 降到 k 维（sklearn 的 TruncatedSVD 返回 U*S）
svd = TruncatedSVD(n_components=k, n_iter=10, random_state=42)
M_reduced = svd.fit_transform(M)

# 3) 归一化到单位圆，便于看"方向相似"
M_norm = M_reduced / np.linalg.norm(M_reduced, axis=1, keepdims=True)
```

A1 的 sanity check 用的是窗口为 1、两句话的小语料，答案矩阵是**对称**的——这提醒你：`M[i][j]` 与 `M[j][i]` 都要计数（除非你刻意只统计单向）。

为什么要归一化？ 因为 TruncatedSVD 返回 $U\Sigma$，模长反映该主成分上的"能量"；归一化后余弦相似度只反映方向，可视化更接近"语义邻近"。

18.5 评估协议：内在、外在、以及与人类判断的相关

协议	数据	指标	陷阱
类比	Google/Karpathy 类比集	top-1 准确率	必须剔除输入词；训练语料不同不可比
相似度	WordSim353、SimLex-999	Spearman ρ	人类评分噪声；`tiger-tiger` 这类自相似项会抬高相关
分类	NER/POS/情感	下游 F1/准确率	归因难：提升可能来自分类器而非词向量
偏置	WEAT、类比审计	效应量	只能测"关联"，不能证明因果

工程建议：**报告多个数据集 + 多种子 + 置信区间**；单次数值不应被当作结论。

18.6 词向量里的偏见：机制与缓解

机制很直接：语料中"护士—女性""工程师—男性"的共现统计会被目标函数忠实保留。Bolukbasi 等 2016 的著名例子是 `man:computer_programmer::woman:homemaker`。

缓解方法	做法	代价
硬去偏（hard debias）	找性别方向 $\boldsymbol{g}$，把中性词投影掉 $\boldsymbol{g}$ 分量，并等距化	会破坏"性别本身有意义"的词（如 king/queen）
软去偏	只减弱、不完全消除	偏见残留
数据层	重采样/过滤	改变语义分布，可能损害下游
下游约束	在任务层加公平性正则	任务相关、非通用

关键提醒：去偏不是"让向量里没有性别"，而是"让性别不承载不该承载的语义"；这是一个规范性（normative）选择，不是纯技术问题。

18.7 常见误区（专题补充）

1. **误区：GloVe 是"另一种 word2vec"。** 它的目标函数是加权最小二乘拟合 log 共现，不是采样预测；训练动态不同，超参敏感性也不同。
2. **误区：PMI 越大越好。** 低频词的 PMI 方差极大，需要平滑/截断（SPPMI、α–PMI）。
3. **误区：SVD 的 U 就是词向量。** 取 U、$U\Sigma$、$U\Sigma^{1/2}$ 都有人用，下游表现不同；A1 用的是 $U\Sigma$。
4. **误区：词向量评估是"客观"的。** 类比集的构造成分很强（多为英语、男性/女性二元），跨语言/跨文化不成立。

第 19 章 序列模型专题：RNN、LSTM 与梯度病理

> 本章把 L04/L05 的"梯度消失"推到可计算的程度，并给出 LSTM/GRU 的门控对照。CS336 里这些内容被 Transformer 取代，但**梯度病理的诊断方法**是通用的。

19.1 RNN 的 BPTT 与雅可比连乘

$$h_t = \tanh(a_t), \quad a_t = W_{hh}h_{t-1} + W_{xh}x_t + b_h$$

损失 $J = \sum_t J_t$。对 W_{hh} 的梯度：

$$\frac{\partial J}{\partial W_{hh}} = \sum_{t=1}^{T} \frac{\partial J_t}{\partial W_{hh}} = \sum_{t=1}^{T}\sum_{k=1}^{t} \frac{\partial J_t}{\partial h_t}\left(\prod_{i=k+1}^{t} \frac{\partial h_i}{\partial h_{i-1}}\right)\frac{\partial h_k}{\partial W_{hh}}$$

其中

$$\frac{\partial h_i}{\partial h_{i-1}} = \operatorname{diag}\left(1-\tanh^2(a_i)\right) W_{hh}$$

连乘的谱范数被 $\prod_i \|W_{hh}\| \cdot \|1-\tanh^2\|$ 控制：

- 若 $\|W_{hh}\|$ 的最大奇异值 < 1（更准确地说，谱半径加上 $\tanh'$ 的上界 1），梯度指数衰减；
- 若 > 1，指数爆炸。

Pascanu 等 2013 的分析还指出：即使谱半径略大于 1，只要 $\tanh$ 饱和（导数接近 0），梯度仍会消失——**饱和区是消失梯度的另一个来源**。

19.2 梯度裁剪：公式与实现

$$\text{if } \|\hat{g}\|_2 > \tau: \quad \hat{g} \leftarrow \frac{\tau}{\|\hat{g}\|_2}\hat{g}$$

注意是**全局范数**（所有参数拼起来）而非逐参数裁剪；PyTorch：

```
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
```

τ 的常用值 1–5。裁剪只解决爆炸，不解决消失；它也不是"越狠越好"——裁得太狠等于把学习率压小。

19.3 LSTM 的梯度通路

LSTM 的 cell state 更新 $c_t = f_t \odot c_{t-1} + i_t \odot g_t$，因此

$$\frac{\partial c_t}{\partial c_{t-1}} = \mathrm{diag}(f_t) + (\text{通过门控的间接项})$$

忽略间接项时，梯度沿时间传播只需连乘 f_t。若 $f_t \approx 1$（"记住"），梯度几乎无损；若 $f_t \approx 0$（"遗忘"），梯度被切断。**门控让模型自己学习"记多久"**，这是 LSTM 相对 vanilla RNN 的核心优势。

但要注意：$h_t = o_t \odot \tanh(c_t)$，输出通路上仍有 $\tanh'$ 与 o_t，所以 LSTM 不是"无梯度消失"，而是"提供了一条更短的通路"。

GRU 是简化版：把 input/forget 合并为 update gate z_t，并引入 reset gate r_t：

$$\begin{aligned} z_t &= \sigma(W_z[h_{t-1}; x_t]), \qquad & r_t &= \sigma(W_r[h_{t-1}; x_t]) \\ \tilde{h}_t &= \tanh(W_h[r_t \odot h_{t-1}; x_t]) \\ h_t &= (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \end{aligned}$$

参数比 LSTM 少约 1/4，短序列上常相当；长序列/需要精确记忆时 LSTM 仍有优势（经验结论，任务相关）。

19.4 四种"序列建模"路线的对照

路线	长程依赖	并行	位置归纳偏置	代表
n-gram	仅 $n-1$ 步	✅（查表）	无	统计 MT、ARPA
RNN/LSTM	理论无限、实践有限	❌	强（顺序）	ELMo、seq2seq
CNN（因果卷积）	感受野随层数线性增长	✅	中（局部）	WaveNet、CharCNN
Transformer	单层全局	✅	弱（需位置编码）	GPT/BERT/T5

这张表解释了 Transformer 为什么赢：它同时拿到"全局感受野"与"并行"，代价是 $O(n^2)$ 与需要显式位置信息。

19.5 调试序列模型的检查清单

1. **先过拟合一个小 batch**（4–8 条），确认 loss 能到接近 0；
2. **检查梯度范数曲线**：突然尖峰 → 爆炸；长期接近 0 → 消失/死单元；
3. **检查初始 loss**：语言模型初始 loss 应约 $\ln|V|$（均匀分布），若远高说明初始化/标签错位；
4. **检查 teacher forcing 的错位**：输入 `x[:-1]` 对目标 `x[1:]`，差一位是最常见的 bug；
5. **检查 padding 与 mask**：padding token 不应参与 loss，也不应被 attention 注意到；
6. **梯度裁剪 + 学习率 warmup**：RNN/Transformer 都受益；
7. **多种子**：序列模型方差大，单次结果不可信。

19.6 常见误区（专题补充）

1. **误区：梯度消失 = 梯度为 0。** 它是"指数小"，在 float32 下会下溢为 0，但在 log 空间里仍可观测。
2. **误区：LSTM 一定优于 GRU。** 取决于数据规模与任务；GRU 参数少、在小数据上常更稳。

3. **误区：截断 BPTT 只是工程妥协。** 它显式定义了"模型能归因多远"，是建模选择。

4. **误区：双向 RNN 可以用于生成。** 双向需要看到完整序列，生成时未来不可见；encoder 可以双向，decoder 不行。

第 20 章 注意力与 Transformer 专题：形状、复杂度与变体

> 本章是"工程手册"：所有维度、复杂度、显存公式都写成可代入的形式。与 CS336 的 kernel/并行章节互补——这里关心"模型定义"，那里关心"怎么跑快"。

20.1 一张维度表（以 GPT-2 small / BERT-base / Llama-3-8B 为例）

符号	含义	GPT-2 small	BERT-base	Llama-3-8B
L	层数	12	12	32
d	隐维 d_{model}	768	768	4096
h	头数	12	12	32
$d_h = d/h$	每头维度	64	64	128
d_{ff}	FFN 中间维	3072	3072	14336
V	词表	50257	30522	128256
n	上下文长度	1024	512	8192
激活	FFN	GELU	GELU	SwiGLU
归一化	位置	Pre-LN	Post-LN	Pre-LN（RMSNorm）
位置编码	—	可学习绝对	可学习绝对	RoPE

参数量速算（每层）：self-attention $4d^2$（W_Q, W_K, W_V, W_O）+ FFN $2d\,d_{ff}$（两层）+ LN $2d$。GPT-2 small：$12 \times (4 \times 768^2 + 2 \times 768 \times 3072) \approx 12 \times (2.36M + 4.72M) \approx 85M$，加 embedding $50257 \times 768 \approx 38.6M$，合计约 124M——与公开的 GPT-2 small（约 124M）一致；课程幻灯片里的 117M 指 **GPT-1**，两者常被混淆。

20.2 复杂度与显存

操作	计算量	激活显存	备注
QKV 投影	$3 \times 2nd^2$ FLOPs	$O(nd)$	与 n 线性
$QK^\top$	$2n^2d$	$O(n^2)$（每头）	二次瓶颈
softmax	$O(n^2)$	$O(n^2)$	可用 online softmax 省显存
$\text{Attn} \cdot V$	$2n^2d$	$O(nd)$	二次
FFN	$2 \times 2nd\,d_{ff}$	$O(nd_{ff})$	$d_{ff} \approx 4d$ 时占大头
全模型前向	$\approx 2N_{\text{param}} \cdot n$（每 token 约 $2N$）	—	训练约 $6N$ FLOPs/token

KV cache（推理时缓存历史 K/V）：

$$\text{bytes} = 2 \times L \times n \times h \times d_h \times \text{bytes per element}$$

以 Llama-3-8B（$L = 32, h = 32, d_h = 128$）、$n = 8192$、fp16 为例：$2 \times 32 \times 8192 \times 32 \times 128 \times 2 \approx 4.3$ GB。这就是 MQA/GQA 的动机：让多个 query 头共享同一组 K/V。

变体	K/V 头数	KV cache	质量
MHA	h	1×	基线
MQA	1	$1/h$	略降，推理快
GQA	g（如 $h/8$）	g/h	接近 MHA，工业默认

20.3 Pre-LN vs Post-LN

结构	公式	优点	缺点
Post-LN（原论文）	$x \leftarrow \text{LN}(x + \text{Sublayer}(x))$	表达力强	深层需 warmup，易不稳
Pre-LN	$x \leftarrow x + \text{Sublayer}(\text{LN}(x))$	训练稳定、可用大学习率	表达略弱；需 final LN
RMSNorm	除以 RMS，不减均值	更省算力	与 LN 略有差异

"Pre-LN 更稳"的直觉：残差主干上没有任何归一化，梯度可以"直通"到底；Post-LN 每层都改写主干，深层容易梯度失配。

20.4 位置编码的演化

方案	形式	外推	用于
正弦固定	加性、固定频率	理论可以、实际差	原 Transformer
可学习绝对	加性、每位置一个向量	❌	BERT/GPT-2
相对位置（Shaw 2018）	加在 attention logits 上的相对偏置	较好	T5 的 relative bias
RoPE	对 q, k 旋转，点积只依赖相对距离	✅（配合插值）	Llama、Qwen、PaLM
ALiBi	按距离给 attention 加线性惩罚	✅	BLOOM 等

RoPE 的关键性质：$\langle R_m q, R_n k \rangle$ 只依赖 $m - n$，且 $\|R_m q\| = \|q\|$（保范数）。这让"位置 10/11"与"100/101"的注意力分数相同——**相对位置**而非绝对位置。

20.5 一个可运行的 attention 骨架（含形状注释）

```
import torch, torch.nn as nn, torch.nn.functional as F

class CausalSelfAttention(nn.Module):
    def __init__(self, d=768, h=12, n_ctx=1024, pdrop=0.1):
        super().__init__()
        assert d % h == 0
        self.h, self.dh = h, d // h
        self.qkv = nn.Linear(d, 3 * d)          # [B, T, d] -> [B, T, 3d]
        self.proj = nn.Linear(d, d)
        self.attn_drop = nn.Dropout(pdrop)
        self.resid_drop = nn.Dropout(pdrop)
        # 上三角为 1 的布尔 mask，注册为 buffer（不参与训练）
        self.register_buffer("mask", torch.triu(torch.ones(n_ctx, n_ctx), diagonal=1).bool())

    def forward(self, x):                        # x: [B, T, d]
        B, T, C = x.shape
        q, k, v = self.qkv(x).split(C, dim=2)    # 各 [B, T, d]
        # [B, T, h, dh] -> [B, h, T, dh]
        q = q.view(B, T, self.h, self.dh).transpose(1, 2)
        k = k.view(B, T, self.h, self.dh).transpose(1, 2)
        v = v.view(B, T, self.h, self.dh).transpose(1, 2)
        att = (q @ k.transpose(-2, -1)) / (self.dh ** 0.5)   # [B, h, T, T]
        att = att.masked_fill(self.mask[:T, :T], float("-inf"))
        att = F.softmax(att, dim=-1)             # 未来位置权重为 0
        att = self.attn_drop(att)
        y = att @ v                              # [B, h, T, dh]
        y = y.transpose(1, 2).contiguous().view(B, T, C)
        return self.resid_drop(self.proj(y))
```

三个最容易错的点：`transpose` 后要 `contiguous()` 才能 `view`；mask 的方向（`triu(diagonal=1)` 遮住严格上三角）；缩放用 d_h 而非 d。

20.6 常见误区（专题补充）

1. **误区：把 d 当缩放因子。** 缩放是 $\sqrt{d_h}$（每头维度），不是 $\sqrt{d_{model}}$。
2. **误区：mask 加 0/1 乘法。** 常见错误是 `att * mask` 把未来位置乘 0，但 softmax 之前乘 0 不等于概率 0（分母仍包含它们）；正确做法是加 $-\infty$。
3. **误区：KV cache 只省计算不省显存。** 它省的是重复前向，代价是显存随 n 线性增长。
4. **误区：FlashAttention 改变了数学。** 它是 IO 感知的精确注意力（分块 + online softmax），结果与标准注意力一致（浮点误差内）。
5. **误区：位置编码越复杂越好。** 关键是"训练长度 vs 推理长度"的匹配；RoPE + 插值是目前最实用的组合。

第 21 章 预训练与微调专题：BERT/GPT/T5 与适配配方

> 本章把第 8–10 章压成"选型 + 配方"两张表，并补上课程幻灯片未展开的微调超参与灾难性遗忘。

21.1 三种架构的对照（含目标函数与典型用法）

维度	BERT（encoder）	GPT（decoder）	T5（encoder-decoder）
目标	Masked LM（15%，80/10/10）+ NSP（后被弃）	Causal LM	Span corruption（哨兵占位）
上下文	双向	单向（左）	encoder 双向 / decoder 单向
典型任务	分类、NER、抽取式 QA、句对	生成、对话、few-shot	翻译、摘要、生成式 QA
微调方式	加分类头，全参或 LoRA	加 LM 头（或分类头）	seq2seq 目标
优势	表示强、样本效率高	通用、可 zero/few-shot	输入输出都是序列时最自然
弱点	不能自然生成；`[MASK]` 不一致	不能看右侧上下文	结构复杂、推理成本高

一个常被忽略的点：GPT 类模型做分类时，用**最后一个 token 的隐状态**接线性层（DFP 的 `GPT2SentimentClassifier` 就是这样）；BERT 类用 `[CLS]`。选择取决于"哪个位置的表示聚合了全局信息"——因果模型只有最后位置能看到全句。

21.2 微调配方（经验值，需按任务调）

超参	分类（BERT/GPT）	生成（T5/GPT）	LoRA
学习率	2×10^{-5}–5×10^{-5}	1×10^{-5}–3×10^{-5}	1×10^{-4}–3×10^{-4}
batch	16–32	32–128（按 token 计）	同全参
epochs	3–5	1–3	3–10
warmup	6%–10%	6%–10%	0–6%
weight decay	0.01	0.01	0（LoRA 常不加）
其他	dropout 0.1、梯度裁剪 1.0	label smoothing 0.1	$r=8$–32、$\alpha=16$–32

小数据的关键：先用小学习率跑 1 个 epoch 看 dev，再决定是否加 epoch；分类任务上"全参微调 + 小 lr"往往比"大 lr + 少 epoch"稳。

21.3 灾难性遗忘与"微调把模型弄坏"

现象：在窄领域数据上全参微调后，模型在原任务/通用指令遵循上明显退化（第 13 章 L13 提到"naïve midtraining 会导致 instruction following 退化"）。

缓解	机制	代价
低学习率 + 少 epoch	少改参数	可能欠拟合
LoRA/Adapter	只改低秩/小模块	容量受限
数据混合	保留 10%–30% 通用数据	需要通用数据
KL 正则（RLHF）	惩罚偏离参考模型	需要参考模型
同策略蒸馏	用学生自己的上下文 + teacher 分布	需要 teacher

21.4 PPO / DPO / GRPO 的选型表

维度	PPO（RLHF）	DPO	GRPO
需要 reward model	✅	❌（隐式）	可选（可用规则奖励）
需要 value network	✅	❌	❌
数据	在线采样	离线偏好对	在线采样（组内比较）
稳定性	低（超参敏感）	高	中
成本	高（4–5 个模型）	低	中
适合	有 RM、追求上限	有高质量偏好对、快速迭代	可验证任务（数学/代码）

21.5 常见误区（专题补充）

1. **误区：BERT 的 `[CLS]` 天然是句向量。** 它只是被 NSP 训练过；不做 sentence-BERT 之类的对比学习，直接取 `[CLS]` 做相似度效果很差。
2. **误区：全参微调一定优于 LoRA。** 在数据少、任务窄时 LoRA 常更稳（低秩正则）；数据多、任务与预训练差异大时全参更强。
3. **误区：SFT 能注入新知识。** 它主要教"格式与行为"；新事实靠 continued pretraining 或 RAG。
4. **误区：DPO 不需要参考模型。** DPO 的损失里显式出现 p_{PT}，必须有冻结的参考模型（通常就是 SFT 后的模型）。

第 22 章 评测专题：指标公式、口径与选择

> 本章把第 12 章的指标写成可计算形式，并给出"什么任务用什么指标"的决策表。所有公式都可手算验证。

22.1 困惑度：定义、口径与陷阱

$$\mathrm{PPL} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log p_\theta(w_i \mid w_{<i})\right)$$

三条口径规则：

1. **按 token 还是按词**：分母不同，PPL 不同；跨 tokenizer 不可比。
2. **是否包含特殊 token**（`<s>`、`</s>`、padding）：包含会改变数值。
3. **是否用 teacher forcing**：PPL 永远用真实前缀（不是模型自己生成的）。

为什么 PPL 不能跨模型比较：词表越大，每 token 的预测越"容易"（候选多但概率质量分散），PPL 的量纲随分词变化。正确做法是报告 bits per character（BPC） 或 bits per byte：

$$\mathrm{BPC} = \frac{-\sum_i \log_2 p_\theta(w_i \mid w_{<i})}{\text{字符数}}$$

22.2 BLEU：公式 + 手算例子

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \log p_n\right), \quad \text{BP} = \begin{cases} 1 & c > r \\ e^{1-r/c} & c \le r \end{cases}$$

其中 $p_n = \frac{\sum_{\text{gram}} \min(\text{count}_{\text{cand}}, \text{count}_{\text{ref}})}{\sum_{\text{gram}} \text{count}_{\text{cand}}}$。

手算：参考 `the cat sat on the mat`，候选 `the cat is on the mat`（$c = r = 6$）。

阶	候选 n-gram 数	匹配数	p_n
1	6	5（the×2, cat, on, mat）	5/6
2	5	3（the cat, on the, the mat）	3/5
3	4	1（on the mat）	1/4
4	3	0	0

于是 BLEU-4 = 0（只要有一阶为 0，几何平均就为 0）；BLEU-2 $= \sqrt{(5/6)(3/5)} = \sqrt{0.5} \approx 0.707$。这个例子说明两件事：**BLEU 对短句极不稳定**，且**高阶 n-gram 的 0 匹配会直接归零**。实践中常报告 BLEU-4 并配合 chrF/BERTScore。

22.3 ROUGE：召回视角

变体	定义	用途
ROUGE-N	参考 n-gram 被候选覆盖的比例（召回）	摘要
ROUGE-L	基于最长公共子序列的 P/R/F	摘要、长文
ROUGE-W	加权 LCS（连续匹配加分）	摘要

对上例：ROUGE-1 $= 5/6$，ROUGE-2 $= 3/5$，ROUGE-L：LCS 为 `the cat on the mat`（5 个 token），$R = P = F = 5/6$。

BLEU vs ROUGE：BLEU 惩罚"多说的"（精度），ROUGE 惩罚"漏说的"（召回）。摘要任务两者都报，是因为"简洁"与"覆盖"是两个不同的失败模式。

22.4 chrF 与字符级指标

$$\text{chrF}_\beta = (1+\beta^2)\frac{P \cdot R}{\beta^2 P + R}$$

其中 P, R 是字符 n-gram（通常 $n = 1..6$）的精度与召回，$\beta = 3$ 时更看重召回。chrF 对形态变化、黏着语、拼写差异更稳健，DFP 的十四行诗生成就用它（因为诗的评价不该被分词边界主导）。

22.5 排序/选择类指标：pass@k 与 Elo

pass@k（代码生成）：采样 n 个解，其中 c 个正确，n 个中至少一个正确的无偏估计：

$$\text{pass@}k = \mathbb{E}_{\text{problems}}\left[1 - \frac{\binom{n-c}{k}}{\binom{n}{k}}\right]$$

注意：直接用 $1-(1-c/n)^k$ 是有偏的；代码库（HumanEval）用的是组合数公式。

Elo / Bradley–Terry（Arena 排名）：

$$P(A \succ B) = \frac{1}{1 + 10^{(R_B - R_A)/400}}$$

每场比赛后按 $\Delta R = K(S_{AB} - P(A \succ B))$ 更新。它的假设是"偏好可由单一标量实力解释"——对风格差异大的模型未必成立。

22.6 指标选择决策表

任务	首选	备选	不要用
机器翻译	BLEU-4 + chrF + COMET	BERTScore	单一 BLEU 作结论
摘要	ROUGE-1/2/L + BERTScore	chrF、人工评分	只看 ROUGE-2
开放域 QA	Exact Match / F1	LLM judge	BLEU
分类/NER	准确率 / F1（宏/微）	AUC	PPL
语言建模	PPL / BPC（同分词）	bits-per-byte	跨 tokenizer 比 PPL
对话/写作	成对人工或 LLM judge + rubric	Arena Elo	单一自动指标
代码	pass@k + 单元测试	人工审查	BLEU

22.7 评测的"元"问题

1. **污染**：训练集含测试题。检测用 n-gram 重叠（8–13 gram）+ 近重复；但闭源模型无法验证。
2. **提示敏感**：同一模型换 prompt 可差十几个点；报告必须固定 prompt 与抽取脚本。
3. **选择偏差**：只报最好的 seed/checkpoint；应报均值±方差。
4. **Goodhart**：指标一旦成为目标就失真；需要"动态/对抗"基准。
5. **效度**：指标测的是"与人类判断的相关性"，不是"能力本身"；相关性要在**目标分布**上验证。

22.8 常见误区（专题补充）

1. **误区：BLEU 可以跨语言比较。** 不同语言的分词/形态差异巨大，BLEU 量纲不同。
2. **误区：ROUGE 高 = 摘要好。** 抽取式拼接常拿高 ROUGE 但可读性差；需要人工/LLM judge 补充。
3. **误区：pass@k 越大越好。** 它依赖采样温度与 n；不同 n 下不可比。
4. **误区：Elo 是绝对能力。** 它只在同一批对局内相对有效；新模型加入会改变基线。
5. **误区：LLM judge 可以完全替代人工。** 它便宜、可复现，但有自我/长度偏好；高 stakes 评测需要人机结合。

第 23 章 与已有笔记的衔接：三份笔记怎么一起用

23.1 知识点映射表

本笔记章节	CS336 对应讲次	CS229 对应内容	建议读法
第 3 章 词向量	第 1 讲（tokenization 只讲 BPE 实现）	无	先本笔记的"为什么"，再看 CS336 的 BPE 代码
第 4 章 反向传播	第 2 讲（PyTorch/FLOPs）	第 1–2 章的梯度下降、矩阵求导	手推一遍 A2 的 1(b)，再用 CS336 的 FLOPs 视角算成本
第 5 章 RNN/LSTM	第 3 讲（架构）只把 RNN 当历史	序列模型/EM 无直接对应	重点是梯度病理，CS336 不展开
第 6 章 Transformer	第 3–4 讲（架构、attention 替代、MoE）	无	本笔记管"定义与形状"，CS336 管"怎么跑快"
第 8 章 预训练	第 9、11 讲（scaling laws）	无	本笔记管"目标函数与数据"，CS336 管"拟合 scaling law"
第 9 章 后训练	第 12 讲（RL 基础在 CS234）	强化学习（CS229 第 3 部分）	DPO 推导自己推一遍；PPO/GRPO 对照 CS234
第 10 章 PEFT	第 10 讲（推理显存）	无	LoRA 的显存收益可用 CS336 的 KV cache/参数公式估算
第 12/22 章 评测	第 12 讲（评测）	假设检验/置信区间	指标公式看本笔记，统计显著性看 CS229
第 20 章 专题	第 2–7 讲（FLOPs、GPU、kernel、并行）	无	维度表 + 复杂度公式是两边共同的"接口"

23.2 三条学习路线

路线 A：建模优先（推荐给要做 NLP 研究的人） 第 1→2→3→4→5→6 章（逐讲精读）→ 第 18–20 章（专题推导）→ 第 8–10 章（预训练/后训练/PEFT）→ 第 12、22 章（评测）→ 做 A1–A3。 预计 3–4 周（每天 2 小时），重点是能手推 softmax 梯度、attention 形状、DPO 损失。

路线 B：系统优先（已有 CS336 基础的人） 第 1 章 → 第 6、20 章（快速对齐术语与形状）→ 第 8–11 章（预训练/后训练/PEFT/RAG）→ 第 13–14 章（推理系统）→ 第 12、22 章（评测）。 跳过第 2–5 章的详细推导，只读"常见误区"。

路线 C：面试/速查（1–2 天） 第 18.1（word2vec 梯度）、第 20.1–20.2（维度/复杂度）、第 21.4（PPO/DPO/GRPO 选型）、第 22.2–22.6（指标公式与选择表）、第 24 章（自测题）。

23.3 不要重复造的部分

- **tokenizer 实现**：CS336 第 1 讲有完整 BPE 训练/编码代码；本笔记只讲"为什么词表大小影响多语言公平"。
- **训练框架**：分布式、混合精度、激活重计算、FlashAttention kernel 在 CS336；本笔记只给"模型定义"层面的形状。
- **scaling law 拟合**：CS336 第 9、11 讲有拟合方法；本笔记只引用结论（如"70B 在更优配比下超过更大模型"）。
- **RL 算法细节**：CS234 有 PPO/GAE 的完整推导；本笔记只讲"LLM 场景下的工程取舍"。

第 24 章 复习清单与自测题（附答案）

24.1 复习清单（能全部打勾再进下一章）

词向量 – [ ] 能写出 skip-gram 的似然、负对数似然、softmax 形式 – [ ] 能推出 $\partial J/\partial v_c = U(\hat{y}-y)$，并解释两项的直觉 – [ ] 能解释为什么负采样用 $U(w)^{3/4}$、为什么两个向量 – [ ] 能说清 GloVe 的加权最小二乘与 PMI 的关系 – [ ] 能解释内在/外在评估的取舍，以及类比评估的缺陷

序列模型 – [ ] 能写出 RNN/LSTM 的前向公式与门控作用 – [ ] 能解释梯度消失的连乘来源与裁剪的作用 – [ ] 能说清 teacher forcing 与 exposure bias – [ ] 能用 PPL 的定义解释"为什么不能跨 tokenizer 比较"

Transformer – [ ] 能默写 scaled dot-product attention 与多头公式 – [ ] 能说清 mask、位置编码、FFN、残差、LayerNorm 各自解决什么 – [ ] 能算 GPT-2 small 的参数量与 KV cache 大小 – [ ] 能解释 Pre-LN/Post-LN、MHA/MQA/GQA 的取舍

预训练/后训练 – [ ] 能对比 BERT/GPT/T5 的目标函数与适用任务 – [ ] 能写出 BERT 的 15%/80/10/10 并解释为什么 – [ ] 能推出 DPO 损失（从 KL 正则目标到 Bradley-Terry） – [ ] 能说清 PPO/DPO/GRPO 的模型数量与数据来源差异

评测 – [ ] 能手算 BLEU-2/BLEU-4 与 ROUGE-1/2/L – [ ] 能说出 BLEU 与 ROUGE 的偏向差异 – [ ] 能列出三种 benchmark 虚假偏置与对应案例 – [ ] 能解释 LLM judge 的三种偏差与缓解

24.2 自测题（18 题，答案在后）

1. 为什么 one-hot 向量无法表达"motel 与 hotel 相似"？word2vec 如何解决？
2. skip-gram 中每个词为什么有两个向量？训练完怎么合并？
3. 负采样把 $|V|$ 类 softmax 变成什么？采样分布为什么是 $U^{3/4}$？
4. 写出 GloVe 的目标函数，并解释 $f(X_{ij})$ 的作用。
5. 交叉熵与负对数似然在 one-hot 标签下是什么关系？
6. 为什么"没有非线性，多层网络等价于一层"？
7. 写出 RNN 的隐状态更新，并说明 BPTT 中梯度为什么会消失。
8. 梯度裁剪的公式是什么？它解决哪个问题、不解决哪个问题？
9. LSTM 的哪条通路让梯度可以"直通"？为什么不是完全无消失？
10. 写出 scaled dot-product attention；为什么除以 $\sqrt{d_k}$？
11. self-attention 为什么需要位置编码？正弦编码的两个优缺点是什么？
12. Pre-LN 与 Post-LN 的公式差异？哪个更稳定？
13. BERT 的 mask 为什么要 80/10/10？NSP 为什么后来被弃用？
14. T5 的 span corruption 与 BERT 的 MLM 有什么本质差异？
15. 写出 DPO 的损失，并说明 $\log Z(x)$ 为什么消失。
16. PPO 为什么需要 value network？GRPO 如何去掉它？
17. 手算：参考 `the cat sat on the mat`，候选 `the cat is on the mat`，求 BLEU-2、ROUGE-1、ROUGE-L。
18. 为什么"校准的语言模型必须幻觉"？这与 Good-Turing 有什么关系？

24.3 答案

1. one-hot 向量两两正交，余弦相似度恒为 0，无法表达相似性；word2vec 用低维稠密向量 + 分布语义目标，让"上下文相似"变成"向量点积大"。

2. 因为 $u^\top v$ 对称，若只用一个向量，词与自身的相似度会和与其他词的相似度混在一起，优化更难；训练完通常把中心向量与上下文向量相加或平均。

3. 变成"正样本 vs K 个负样本"的二分类（logistic）；$U^{3/4}$ 把分布压平，提升低频词被采为负样本的概率，避免负样本几乎全是高频虚词。

4. $J=\sum_{i,j} f(X_{ij})(w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$；$f(x)=\min(1,(x/x_{\max})^\alpha)$ 降低低频共现的权重（噪声大）、限制高频共现的权重。

5. 在 one-hot 下 $H(p,q) = -\sum_i p_i \log q_i = -\log q_y$，正好等于真实类的负对数似然。

6. 若无非线性，$W_2(W_1 x) = (W_2 W_1)x$，多层复合仍是线性变换，表达力不超过单层。

7. $h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$；BPTT 中 $\partial h_t/\partial h_{t-k}$ 是雅可比连乘，谱范数 <1 时指数衰减（$\tanh$ 饱和会加剧）。

8. $\hat{g} \leftarrow \tau \hat{g}/\|\hat{g}\|$（当范数超过 τ）；解决爆炸，不解决消失。

9. cell state 的加法更新 $c_t = f_t \odot c_{t-1} + i_t \odot g_t$，$\partial c_t/\partial c_{t-1} = \mathrm{diag}(f_t)$；但输出通路 $h_t = o_t \odot \tanh(c_t)$ 仍有 $\tanh'$ 与门控，所以只是"更短的通路"。

10. $\mathrm{Attention} = \mathrm{softmax}(QK^\top/\sqrt{d_k})V$；点积方差随 d_k 增大，不缩放会让 softmax 饱和、梯度趋零。

11. self-attention 对输入置换等变，本身不含顺序信息；正弦编码优点是固定、不占参数、理论上可外推，缺点是实际外推效果差且不可学习。

12. Post-LN：$x \leftarrow \mathrm{LN}(x + \mathrm{Sublayer}(x))$；Pre-LN：$x \leftarrow x + \mathrm{Sublayer}(\mathrm{LN}(x))$。Pre-LN 更稳定（残差主干无归一化，梯度直通），现代大模型多用。

13. 80% `[MASK]`、10% 随机词、10% 不变，是为了不让模型只学 `[MASK]` 的表示、逼迫它对所有词建立表示；NSP 被 RoBERTa 去掉后效果更好，说明它不必要。

14. MLM 遮单个 token（可独立预测），span corruption 遮连续 span 并用哨兵占位、由 decoder 生成被遮 span；后者是 seq2seq 式的去噪目标。

15. $J_{DPO} = -\mathbb{E}\log\sigma\left(\beta\log\frac{p_\theta(y_w)}{p_{PT}(y_w)} - \beta\log\frac{p_\theta(y_l)}{p_{PT}(y_l)}\right)$；$\log Z(x)$ 只依赖 x，在两个奖励相减时抵消。

16. PPO 用 value network 估计基线以算 GAE、降低方差；GRPO 用同一 prompt 的一组回答的组内标准化奖励作为 advantage，不需要 critic。

17. BLEU-2 $= \sqrt{(5/6)(3/5)} = \sqrt{0.5} \approx 0.707$；ROUGE-1 $= 5/6 \approx 0.833$；ROUGE-L 的 LCS 为 `the cat on the mat`，$R = P = F = 5/6$。

18. 任意事实无法从规则推出，训练数据不可能覆盖全部真值；校准模型必须给未见事实留概率质量，这部分质量也会分给"看似合理但为假"的陈述。Good-Turing 的"未见事件总概率≈单次事件总概率"是同一思想。

第 25 章 参考来源

> 以下 URL 均在 2026-11 访问并核实可下载/可打开；标注"未公开"的是我按命名规律探测返回 404 的项，不作为事实来源。

25.1 课程与作业（一手）

内容	URL	访问日期
CS224n 主页（Winter 2026）	https://web.stanford.edu/class/cs224n/	2026-11
当季日程锚点	https://web.stanford.edu/class/cs224n/index.html#schedule	2026-11
办公时间/ staff	https://web.stanford.edu/class/cs224n/office_hours.html	2026-11
期末项目页	https://web.stanford.edu/class/cs224n/project.html	2026-11
2024 公开视频播放列表	https://www.youtube.com/playlist?list=PLoROMvodv4rOaMFbaqxPDoLWjDaRAdP9D	2026-11

25.2 讲义 PDF（`slides_w26/`，全部实测 HTTP 200）

讲次	文件
L01 intro	https://web.stanford.edu/class/cs224n/slides_w26/cs224n-2026-lecture01-intro.pdf
L01 history	https://web.stanford.edu/class/cs224n/slides_w26/cs224n-2026-lecture01-history.pdf
L02 word vectors	https://web.stanford.edu/class/cs224n/slides_w26/cs224n-2026-lecture02-wordvecs.pdf
L03 NN foundations	https://web.stanford.edu/class/cs224n/slides_w26/cs224n-2026-lecture03-neuralnets.pdf
L04 LM & RNN	https://web.stanford.edu/class/cs224n/slides_w26/cs224n-2026-lecture04-rnnlm.pdf
L05 attention & Transformer	https://web.stanford.edu/class/cs224n/slides_w26/cs224n-2026-lecture05-transformers.pdf
L06 final project tips	https://web.stanford.edu/class/cs224n/slides_w26/cs224n-2026-lecture06-final-project.pdf
L07 pretraining	https://web.stanford.edu/class/cs224n/slides_w26/cs224n-2026-lecture07-pretraining.pdf
L08 post-training	https://web.stanford.edu/class/cs224n/slides_w26/cs224n-2026-lecture08-posttraining.pdf
L09 PEFT	https://web.stanford.edu/class/cs224n/slides_w26/cs224n-2026-lecture09-peft.pdf
L10 RAG & agents	https://web.stanford.edu/class/cs224n/slides_w26/cs224n-2026-lecture10-rag-agents.pdf
L11 evaluation	https://web.stanford.edu/class/cs224n/slides_w26/cs224n-2026-lecture11-evaluation.pdf
L12 reasoning 1	https://web.stanford.edu/class/cs224n/slides_w26/cs224n-2026-lecture12-reasoning-part1.pdf
L13 reasoning 2	https://web.stanford.edu/class/cs224n/slides_w26/cs224n-2026-lecture13-reasoning-part2.pdf
L14 tokenization guest	https://web.stanford.edu/class/cs224n/slides_w26/cs224n-2026-lecture14-guest-julie-tokenization-multilinguality.pdf
L16 impact	https://web.stanford.edu/class/cs224n/slides_w26/cs224n-2026-lecture16-impact-on-humanity.pdf
L19 open questions	https://web.stanford.edu/class/cs224n/slides_w26/cs224n-2026-lecture19-open-questions.pdf

未公开（探测 404）：`lecture15-interpretability.pdf`、`lecture17-multimodality.pdf`、`lecture18-tinker.pdf`。L15 在主页 HTML 中链接到的是 2024 年旧讲 `slides/cs224n-spr2024-lecture15-life-after-dpo-lambert.pdf`（Lambert，讲题《Life after DPO》），不是 Been Kim 的可解释性讲。

25.3 作业与 handout

内容	URL	访问日期
A1 起始代码	https://web.stanford.edu/class/cs224n/assignments_w26/a1.zip	2026-11
A2 起始代码	https://web.stanford.edu/class/cs224n/assignments_w26/a2.zip	2026-11
A2 handout	https://web.stanford.edu/class/cs224n/assignments_w26/a2.pdf	2026-11
A3 handout	https://web.stanford.edu/class/cs224n/assignments_w26/a3.pdf	2026-11
A4 handout	https://web.stanford.edu/class/cs224n/assignments_w26/a4.pdf	2026-11
DFP（GPT-2）handout	https://web.stanford.edu/class/cs224n/project_w25/CS_224n__Default_Final_Project__Build_GPT_2.pdf	2026-11
DFP 代码仓库	https://github.com/cfifty/public_cs224n_gpt	2026-11
项目提案说明	https://web.stanford.edu/class/cs224n/project/Project_Proposal_Instructions.pdf	2026-11
项目里程碑说明	https://web.stanford.edu/class/cs224n/project/Project_Milestone_Instructions.pdf	2026-11

25.4 教材与公开读物

内容	URL	访问日期
SLP3 主页（Aug 19, 2026 版）	https://web.stanford.edu/~jurafsky/slp3/	2026-11
SLP3 合订本	https://web.stanford.edu/~jurafsky/slp3/ed3book_aug26.pdf	2026-11
SLP3 Ch.2 Words and Tokens	https://web.stanford.edu/~jurafsky/slp3/2.pdf	2026-11
SLP3 Ch.3 N-gram LM	https://web.stanford.edu/~jurafsky/slp3/3.pdf	2026-11
SLP3 Ch.5 Embeddings	https://web.stanford.edu/~jurafsky/slp3/5.pdf	2026-11
SLP3 Ch.6 Neural Networks	https://web.stanford.edu/~jurafsky/slp3/6.pdf	2026-11
SLP3 Ch.7 Transformers & Pretraining	https://web.stanford.edu/~jurafsky/slp3/7.pdf	2026-11
SLP3 Ch.8 Post-training	https://web.stanford.edu/~jurafsky/slp3/8.pdf	2026-11
SLP3 Ch.9 Masked LMs	https://web.stanford.edu/~jurafsky/slp3/9.pdf	2026-11
SLP3 Ch.11 IR & RAG	https://web.stanford.edu/~jurafsky/slp3/11.pdf	2026-11
SLP3 Ch.13 MT	https://web.stanford.edu/~jurafsky/slp3/13.pdf	2026-11

内容	URL	访问日期
SLP3 Ch.14 RNNs & LSTMs	https://web.stanford.edu/~jurafsky/slp3/14.pdf	2026-11
SLP3 Ch.20 Dependency Parsing	https://web.stanford.edu/~jurafsky/slp3/20.pdf	2026-11
SLP3 附录 C（Kneser-Ney）	https://web.stanford.edu/~jurafsky/slp3/C.pdf	2026-11
SLP3 附录 J（PPMI）	https://web.stanford.edu/~jurafsky/slp3/J.pdf	2026-11

> **编号提示**：课程主页把 "Chapter 9" 标为《The Transformer》，那是 2024 版的编号；2026-08 版里 Ch.7 才是 Transformer，Ch.9 是 Masked LM。引用时以现行 PDF 的标题为准。

25.5 关键论文（本笔记引用到的论断）

主题	论文	URL
word2vec	Mikolov et al., 2013, *Efficient Estimation of Word Representations in Vector Space*	https://arxiv.org/pdf/1301.3781.pdf
负采样	Mikolov et al., 2013, *Distributed Representations of Words and Phrases...*	https://papers.nips.cc/paper/5021-distributed-representations-of-words-and-phrases-and-their-compositionality.pdf
GloVe	Pennington et al., 2014	https://nlp.stanford.edu/pubs/glove.pdf
词向量评估	Levy et al., 2015, *Improving Distributional Similarity with Lessons Learned...*	https://aclanthology.org/Q15-1016/
PMI 理论	Levy & Goldberg, 2014, *Neural Word Embedding as Implicit Matrix Factorization*	https://papers.nips.cc/paper/5477-neural-word-embedding-as-implicit-matrix-factorization.pdf
梯度病理	Pascanu et al., 2013, *On the difficulty of training Recurrent Neural Networks*	https://arxiv.org/pdf/1211.5063.pdf
Attention	Vaswani et al., 2017, *Attention Is All You Need*	https://arxiv.org/abs/1706.03762
LayerNorm	Ba et al., 2016	https://arxiv.org/pdf/1607.06450.pdf
BERT	Devlin et al., 2018	https://arxiv.org/pdf/1810.04805.pdf
GPT-3	Brown et al., 2020	https://arxiv.org/abs/2005.14165
T5	Raffel et al., 2018	https://arxiv.org/abs/1910.10683
LoRA	Hu et al., 2021	https://arxiv.org/abs/2106.09685
QLoRA	Dettmers et al., 2023	https://arxiv.org/abs/2305.14314
DPO	Rafailov et al., 2023	https://arxiv.org/abs/2305.18290
InstructGPT	Ouyang et al., 2022	https://arxiv.org/abs/2203.02155
CoT	Wei et al., 2022	https://arxiv.org/abs/2201.11903

主题	论文	URL
Self-Consistency	Wang et al., 2022	https://arxiv.org/abs/2203.11171
ReAct	Yao et al., 2022	https://arxiv.org/abs/2210.03629
RAG	Lewis et al., 2020	https://arxiv.org/abs/2005.11401
DPR	Karpukhin et al., 2020	https://arxiv.org/abs/2004.04906
DeepSeek-R1	DeepSeek-AI, 2025	https://arxiv.org/abs/2501.12948
DAPO	ByteDance Seed et al., 2025	https://arxiv.org/abs/2503.14476
RoPE	Su et al., 2021	https://arxiv.org/abs/2104.09864
投机解码	Leviathan et al., 2022	https://arxiv.org/abs/2211.17192
Test-time scaling	Snell et al., 2024	https://arxiv.org/abs/2408.03314
BPE	Sennrich et al., 2016	https://arxiv.org/pdf/1508.07909
chrF	Popovic´, 2015	https://aclanthology.org/W15-3049/
BLEU	Papineni et al., 2002	https://aclanthology.org/P02-1040/
ROUGE	Lin, 2004	https://aclanthology.org/W04-1013/
MMLU	Hendrycks et al., 2021	https://arxiv.org/abs/2009.03300
HELM	Liang et al., 2022	https://arxiv.org/abs/2211.09110
SQuAD	Rajpurkar et al., 2016	https://arxiv.org/abs/1606.05250

25.6 已知的不确定项

1. **三场客座讲次（L15/L17/L18）**：当季无公开 PDF，本笔记只按讲题与主页推荐读物概述，未引用讲者具体观点。
2. **讲次与作业的"当季性"**：主页明确"coursework is tentative and subject to change"；若你在别的学期阅读，请重新抓取日程。
3. **A4 的年份笔误**：A4 handout 首页写 "Winter 2025"，正文与截止日期是 Winter 2026（Feb 19）；本笔记以正文日期为准。
4. **DFP handout 的作业编号**：文中提到 "similar to Assignments 4 and 5"，说明该 handout 写作时存在 A5；当季只发布 A1–A4，DFP 独立计分。
5. **部分论文的精确数字**：如 R1 的"1.5B 蒸馏模型超过更大闭源模型"来自讲者转述，本笔记按"讲者口径"引用，未逐一核对原论文表格。